

Transformer GPU Execution — Placement and Interview Guide

This guide explains what a decoder-only Transformer block does on a GPU during inference. It follows the data path:

```
X
|
├─ normalization (often fused with the next operation)
▼
QKV projection → split/reshape into heads; apply position encoding
▼
Attention scores:  $QK^T / \sqrt{d_{\text{head}}} + \text{mask}$ 
▼
row-wise Softmax
▼
Attention probabilities  $\times V$ 
▼
concatenate heads + output projection
▼
residual + normalization
▼
MLP (up/gate, activation, down) + residual
```

The focus is GPU execution, not just the mathematical graph. A framework may fuse, reorder, tile, or recompute intermediate operations while preserving the same result.

Notation Used Throughout

Symbol	Meaning	Typical shape
B	Batch size	number of sequences
S	Query/input sequence length	prompt tokens during prefill; usually 1 during one decode step
T	Key/value context length	all visible tokens, including cached tokens
D	Model/hidden width	e.g. 4096
H	Number of query heads	e.g. 32
H_{kv}	Number of key/value heads	H for MHA; smaller for GQA/MQA
d_h	Head dimension	normally D/H
D_{ff}	MLP intermediate width	commonly several times D
L	Number of Transformer layers	e.g. 32

Unless stated otherwise, activations are shown as $[B, S, D]$. Implementations often flatten the first two dimensions to $[B \cdot S, D]$ for GEMM.

Input Activations (X)

1. Overview

Definition. X is the matrix of hidden-state vectors entering a Transformer sublayer. At the first layer it comes from token embeddings plus positional information; at later layers it is the previous layer's residual-stream output.

X matters because its shape determines the sizes of almost every downstream GPU operation. During prefill, X contains many tokens and produces large, efficient matrix multiplications. During decode, it normally contains one new token per sequence, so the same layers become narrow matrix-vector-like operations.

Real inference engines store X in GPU global memory, usually in FP16, BF16, or a quantized/low-precision format where supported. Interviewers ask about X to test whether a candidate can connect tensor shapes to memory layout, GEMM dimensions, batching, and residual connections.

2. Core Idea

Think of each token as carrying a D -number scratchpad. The numbers do not directly mean "noun," "past tense," or another fixed feature. Together they form a learned representation that every layer reads and updates.

For $B=2$, $S=3$, and $D=4$:

```
X.shape = [2, 3, 4]

sequence 0: [x00 x01 x02 x03]  token 0
             [x10 x11 x12 x13]  token 1
             [x20 x21 x22 x23]  token 2

sequence 1: [ ... four values ... ] × 3 tokens
```

Step by step:

1. Token IDs index an embedding table, producing vectors of width D .
2. Positional information is added to X or later applied to Q and K , depending on the model.
3. A normalization such as RMSNorm or LayerNorm prepares the values for a projection.
4. The GPU views the tensor as rows of tokens and columns of hidden features.
5. Projection GEMMs consume these rows; residual paths preserve the original X for a later addition.

The important mental shift is that logical shape and physical layout are different ideas. $[B, S, D]$ describes meaning; strides describe where elements live. A contiguous row-major tensor places the D features of one token next to one another.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Shape	One hidden vector per token per sequence.	<code>[8, 512, 4096]</code> .	Derive downstream GEMM shapes.
Layout and strides	Physical address increments for each index. Coalesced access depends on layout.	Last dimension contiguous.	Shape does not guarantee contiguity.
Data type	Controls bytes moved, Tensor Core eligibility, range, and accuracy.	BF16 uses 2 bytes/element.	FP16 vs BF16 range; accumulation may be FP32.
Normalization	Stabilizes the values entering attention/MLP. Often RMSNorm in LLMs.	<code>x / rms(x) * weight</code> .	Pre-norm vs post-norm; fusion opportunity.
Residual stream	Original input is added to sublayer output. It helps optimization and information flow.	<code>y = x + attention(norm(x))</code> .	Do not omit residual dependencies from the execution graph.
Positional information	Makes token order observable. Some models add embeddings; many apply RoPE to <code>Q, K</code> .	Rotate pairs of <code>Q/K</code> features.	Positional encoding may not be stored directly in <code>x</code> .
Padding/ragged batches	Sequences may have different lengths. Padding wastes compute unless packed or paged metadata is used.	lengths <code>[128, 90, 17]</code> .	Masking prevents invalid attention but does not automatically remove work.

4. Real-World Example

An inference server receives four prompts of different lengths. It tokenizes them, retrieves embeddings, and builds either a padded tensor such as `[4, 512, D]` or a packed token buffer with length/offset metadata. The padded version is simpler but performs work on padding. The packed version improves useful-token throughput but needs kernels that understand variable sequence boundaries.

5. Diagrams / Mental Models

```

logical token rows          contiguous memory

(batch 0, token 0) [D values] → x x x x ... x
(batch 0, token 1) [D values] → x x x x ... x
...                          fastest-changing index = hidden feature

```

Phase	Logical X	Flattened GEMM view	GPU consequence
Prefill	$[B, S, D]$	$[B \cdot S, D]$	Large $M=B \cdot S$; good parallelism/reuse
Decode	$[B, 1, D]$	$[B, D]$	Small $M=B$; often bandwidth/launch limited

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is X ?	Hidden states, one D -dimensional vector per token.	Shape and semantic role.	Calling it token IDs.
2. What is its shape?	Usually $[B, S, D]$; libraries may flatten to $[B \cdot S, D]$.	Meaning of every axis.	Confusing S and D .
3. Is X the same at every layer?	No. Each block updates the residual stream.	Layer-to-layer flow.	Treating embeddings as permanent.
4. Why flatten B and S ?	The same linear map applies independently to every token, so both axes form GEMM rows.	$[B \cdot S, D] @ [D, N]$.	Believing tokens get mixed by the projection.
5. Where are tokens mixed?	Primarily in attention; per-token projection and MLP operations do not mix sequence positions.	Token-wise vs sequence-wise work.	Saying every GEMM mixes tokens.
6. Why keep a residual copy?	To add the sublayer result back to its input.	Data dependency and extra memory traffic.	Ignoring the residual read/write.
7. How does dtype affect performance?	Smaller types reduce traffic and may use Tensor Cores, but accuracy/range constraints remain.	Bytes, throughput, accumulation.	"Half precision is always twice as fast."
8. Does a mask make padding free?	No. A mask fixes semantics; kernels may still compute padded positions.	Correctness vs efficiency.	Equating masked with skipped.
9. Why must the last dimension often be contiguous?	Adjacent threads can load adjacent features and GEMM libraries prefer regular aligned layouts.	Coalescing and vector loads.	Saying contiguity changes mathematical shape.
10. How does decode change X ?	Only new-token states enter the block, normally $[B, 1, D]$; history is represented by the KV cache.	Autoregressive execution.	Feeding the full prefix through every step.

7. Deep-Dive Questions

1. **Can two tensors with shape $[B, S, D]$ have different performance?** Yes. Different strides, alignment, padding, dtype, and storage location produce different access patterns even when logical shapes match.
2. **Why might an engine not materialize normalized X ?** A fused RMSNorm-plus-projection kernel can normalize values and immediately feed them into the GEMM path, avoiding an HBM round trip.
3. **What happens to X under tensor parallelism?** Depending on the layer and sharding scheme, it may be replicated across ranks or represent a shard that must be gathered/reduced.
4. **Why can a noncontiguous transpose be cheap initially but expensive later?** Creating a view only changes metadata, but a consumer requiring contiguous data triggers a real copy or a slower strided kernel.
5. **What is the activation-memory difference between training and inference?** Training retains many intermediate tensors for backward; inference can reuse/free buffers aggressively, but decode adds persistent KV-cache state.

8. Comparison Tables

Property	Prefill X	Decode X
Tokens processed per request	Entire prompt/chunk	Usually one new token
GEMM row count	$B \cdot S$	B
Typical bottleneck	Often compute for large shapes	Often weight bandwidth/launch overhead
Historical token states	Computed now	Not re-entered; represented by KV cache
Contiguous tensor	Strided/noncontiguous view	
Regular adjacent addresses	Address determined by strides	
Usually friendly to vectorized/coalesced loads	May be supported but slower	
Can require an explicit copy to create	Transpose/view can be metadata-only	

9. Common Mistakes

- Treating X as a single vector instead of a batch of token vectors.
- Forgetting normalization and residual paths because they are absent from a simplified attention formula.
- Assuming reshape always copies data or transpose never copies data.
- Assuming padding masks eliminate padded computation.
- Multiplying memory count by element count without accounting for dtype bytes.
- Confusing model width D with per-head width d_h .

10. Edge Cases / Special Cases

- $S=0$ should be rejected or handled before launching kernels; a zero-size grid can be invalid or meaningless.

- Ragged batches require lengths/offsets so tokens do not attend across sequence boundaries.
- Quantized activations may need per-token or per-group scales.
- Residual and normalized buffers may alias only if the chosen kernel explicitly supports safe in-place execution.
- Very small $B \cdot S$ produces too little parallel work to fill a GPU.
- Encoder-decoder models have additional encoder states used by cross-attention.

11. How to Explain in Interview

X is the $[batch, tokens, model_width]$ hidden-state tensor entering a Transformer block. Linear layers view it as $[B \cdot S, D]$, while attention later reshapes projected values into heads. During prefill $B \cdot S$ is large and GEMMs are efficient; during decode $S=1$, so the work becomes much more bandwidth-sensitive.

12. Quick Revision Notes

- X : one hidden vector per token; usually $[B, S, D]$.
- Projection view: $[B \cdot S, D]$.
- Token mixing happens in attention, not an ordinary per-token linear projection.
- Logical shape is not physical layout.
- BF16/FP16: commonly 2 bytes per activation; accumulation rules depend on the kernel.
- Decode supplies new tokens only; KV cache represents history.
- Trap: masks ensure correctness but may not save compute.

13. Practice Tasks

1. For $[B, S, D]=[4, 128, 4096]$ BF16, compute element count and bytes.
2. Write a C++ function that converts (b, s, d) to a flat row-major index.
3. Given strides, decide whether a tensor is contiguous.
4. Draw both prefill and decode shapes through one block.
5. Profile a contiguous matrix and its transposed view in a framework.

14. Final Cheat Sheet

Item	Remember
Core definition	Hidden states entering a block, one row per token.
Why it matters	Its shape/layout drives every following kernel.
Most asked	Shape, flattening, dtype, layout, prefill vs decode.
Main comparison	Prefill has many token rows; decode normally has one per sequence.
One-line answer	" X is $[B, S, D]$; linear layers flatten tokens to rows, and attention later reorganizes features into heads."

QKV Projection

1. Overview

Definition. QKV projection applies learned linear maps to each token hidden state:

$$\begin{aligned} Q &= XW_Q + b_Q \\ K &= XW_K + b_K \\ V &= XW_V + b_V \end{aligned}$$

Many implementations concatenate the three weight matrices and perform one GEMM:

$$[Q \mid K \mid V] = X [W_Q \mid W_K \mid W_V]$$

This is used in every standard Transformer attention block. It matters because it is usually one of the major matrix multiplications, it creates the tensors later consumed by attention, and during decode it repeatedly streams a large weight matrix for only a few token rows. Interviewers use it to test GEMM shape reasoning, fusion, head layouts, and MHA/GQA/MQA differences.

2. Core Idea

The same token needs three learned “views”:

- **Query:** what information this token is looking for.
- **Key:** what description this token offers for matching.
- **Value:** what content this token contributes if selected.

Analogy: in a library search, a query is the request, keys are catalog labels, and values are the book contents. The analogy explains roles, but the vectors are learned numerical representations—not literal words.

For $X: [M, D]$, where $M=B \cdot S$, standard multi-head attention uses:

$$\begin{aligned} W_Q, W_K, W_V &: [D, D] \\ Q, K, V &: [M, D] \end{aligned}$$

With fused weights $W_{QKV}: [D, 3D]$, the GPU executes roughly $2 \cdot M \cdot D \cdot 3D$ FLOPs. It then splits, reshapes, and often applies RoPE to Q/K:

$$[M, 3D] \rightarrow Q, K, V \rightarrow [B, H, S, d_h]$$

The projection does not compare tokens. Each output row depends only on the corresponding X row; the later QK^T operation creates token-to-token interaction.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Fused QKV GEMM	Concatenate weights and calculate all outputs together. Reduces launches and reads X once at graph level.	$[M,D]@[D,3D]$.	Same math as three projections, different execution.
Head reshape	Split width into H heads of size d_h .	$D=4096, H=32, d_h=128$.	Reshape/transpose layout for batched attention.
MHA	Query, key, and value use H heads.	$H_{kv}=H$.	Largest KV cache among the three variants.
GQA	Several query heads share one K/V head.	$H=32, H_{kv}=8$.	KV/cache reduction and head mapping.
MQA	All query heads share one K head and one V head.	$H_{kv}=1$.	Lowest KV bandwidth/capacity; possible quality trade-off.
Bias and epilogue	Bias, scaling, quantization, or activation can be applied as GEMM output is written.	GEMM + bias.	Epilogue fusion avoids extra kernels.
Positional transform	RoPE is commonly applied after projection to Q and K, not V.	Pairwise rotations by token position.	Cached keys must already encode the correct position.
Quantized weights	Store weights in fewer bits and dequantize during compute.	INT8/FP8/INT4 weight-only.	Bandwidth savings vs conversion/accuracy.

4. Real-World Example

For a Llama-like layer with $D=4096$, $H=32$, $H_{kv}=8$, and $d_h=128$, Q width is 4096 , but K and V widths are each $8 \cdot 128=1024$. The fused output width is $4096+1024+1024=6144$, not $3D=12288$. During decode with batch $B=16$, the GEMM is $[16, 4096]@[4096, 6144]$. New K/V rows are position-encoded and appended to each layer's KV cache.

5. Diagrams / Mental Models

```

      Q [B,H,S,d_h] → score computation
X [B·S,D] → GEMM → split ┤ K [B,H_kv,S,d_h] → KV cache
                        └ V [B,H_kv,S,d_h] → KV cache

Fused weight columns: |----- Q -----|--- K ---|--- V ---|

```

Projection form	GEMM launches	Logical reads of X	Output width
Three separate projections	3	3	$Q + K + V$

Projection form	GEMM launches	Logical reads of X	Output width
Fused QKV projection	1	1	$Q + K + V$

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why create Q, K, and V?	They separate matching intent, matching labels, and content to aggregate.	Learned projections with distinct roles.	Saying Q/K/V are copied embeddings.
2. Does QKV projection mix tokens?	No; the same linear map is independently applied to each token row.	Token mixing occurs in attention.	Confusing GEMM's reduction over D with sequence mixing.
3. What are the GEMM shapes?	$[B \cdot S, D] @ [D, N_{qkv}] \rightarrow [B \cdot S, N_{qkv}]$.	Correct dimensions.	Using S as reduction dimension.
4. Why fuse the three projections?	Fewer launches, larger GEMM, and potential reduction in activation traffic.	Performance without changing math.	Claiming it reduces learned parameters.
5. What is d_h ?	Per-query-head feature dimension, normally D/H .	Relationship among D, H, d_h .	Setting it to sequence length.
6. How does GQA change shapes?	Q has H heads; K/V have $H_{kv} < H$, shared across groups of query heads.	Cache and bandwidth savings.	Reducing query-head count too.
7. Why is decode projection often bandwidth-sensitive?	Very few input rows reuse each weight, so the GPU streams large weights for little arithmetic reuse.	Arithmetic intensity.	Blaming KV cache for projection traffic.
8. Where is RoPE applied?	Usually to Q and K after projection/reshape and before attention/cache use.	Position-correct Q/K.	Applying it to V by default.
9. Are projection weights shared across layers?	Normally no; every layer has its own learned matrices.	Parameter accounting.	Assuming one global W_{QKV} .
10. Why can alignment matter?	Tensor Core kernels prefer supported, aligned dimensions and layouts.	Hardware-friendly multiples.	Treating any odd dimension as equally efficient.

7. Deep-Dive Questions

1. **Does one fused QKV GEMM always mean one CUDA kernel?** Often, but not necessarily. A compiler/library selects implementations, and reshape/RoPE/cache writes may be separate or further fused.
2. **How does tensor parallelism shard QKV?** Commonly by output columns. Each rank owns a subset of heads, so the projection needs no immediate all-reduce if following attention is sharded compatibly.
3. **Why can MQA/GQA improve decode more than prefill?** Decode repeatedly reads historical K/V; fewer KV heads shrink persistent cache capacity and per-step memory traffic.
4. **Can w_{QKV} be stored physically concatenated?** Yes, and production checkpoints/runtimes often pack or reorder weights into the layout preferred by the GEMM kernel.
5. **What changes under weight-only quantization?** Weight bytes fall, but kernels must apply scales and possibly zero points while accumulating in a wider type; speedup depends on kernel and bottleneck.

8. Comparison Tables

Property	MHA	GQA	MQA
Query heads	H	H	H
KV heads	H	$1 < H_{kv} < H$	1
KV-cache size	Highest	Lower by H/H_{kv}	Lowest
Query expressiveness	Separate Q heads	Separate Q heads	Separate Q heads
Common systems reason	Baseline	Quality/efficiency balance	Maximum decode efficiency
Separate projections	Fused projection		
Simpler conceptual graph	Better launch/amortization opportunity		
Three GEMMs	One larger GEMM		
Easy independent layouts	Requires packed output interpretation		
Same learned arithmetic	Same learned arithmetic		

9. Common Mistakes

- Saying QKV projection itself computes attention.
- Assuming Q, K, and V always have identical widths under GQA/MQA.
- Forgetting bias is model-dependent.
- Treating concatenation as a reduction in parameter count.
- Writing $d_h = H/D$ instead of D/H .
- Forgetting layout conversion or RoPE between projection and score computation.

10. Edge Cases / Special Cases

- Cross-attention obtains Q from decoder states but K/V from encoder states.
- Some models use different query and value head dimensions.
- QKV may be stored interleaved by head rather than as three large contiguous regions.
- Tensor-parallel world size must be compatible with the head/hidden sharding scheme or require uneven/padded partitions.

- Quantized checkpoints may use architecture-specific packed formats.
- During decode, only the new K/V are projected; old K/V come from cache.

11. How to Explain in Interview

QKV projection applies three learned linear maps to every token. Implementations usually concatenate their weights into one GEMM, then reshape the output into heads and apply positional encoding to Q and K. It does not mix tokens; QK^T does. In decode, the small number of input rows makes projection weight bandwidth especially important.

12. Quick Revision Notes

- Formula: $[Q|K|V]=XW_{QKV}$.
- Shape: $[B \cdot S, D] @ [D, N_{qkv}]$.
- MHA: $N_{qkv}=3D$; GQA/MQA make K/V narrower.
- Fuse for fewer launches/larger GEMM, not fewer parameters.
- RoPE usually affects Q/K, not V.
- Decode creates only new-token K/V and appends them to cache.
- Trap: projection mixes hidden features, not token positions.

13. Practice Tasks

1. Derive Q/K/V shapes for $B=8, S=512, D=4096, H=32, H_{kv}=8$.
2. Compute QKV parameter count for MHA and GQA, ignoring bias.
3. Compare bytes read for BF16 and INT8 weights.
4. Implement three CPU projections and verify them against one concatenated projection.
5. Draw a head-to-KV-head mapping for $H=8, H_{kv}=2$.

14. Final Cheat Sheet

Item	Remember
Core definition	Learned token-wise maps create Q, K, and V.
Why it matters	Major GEMM; establishes attention tensors and cache entries.
Most asked	Shapes, fusion, heads, GQA/MQA, decode behavior.
Main comparison	MHA has one KV head per Q head; GQA/MQA share KV heads.
One-line answer	"Project each token into query, key, and value spaces—usually with one packed GEMM—then reshape into heads."

Attention Score Computation (QK^T)

1. Overview

Definition. Attention score computation compares every query with every visible key:

Scores = QK^T / √d_h + mask

For each head, $Q:[S,d_h]$ and $K:[T,d_h]$ produce $Scores:[S,T]$. The score is a similarity logit, not yet a probability. Causal masking makes future positions inaccessible.

This operation is used wherever scaled dot-product attention appears. It matters because prefill can contain $S \cdot T$ score entries per head, while decode has only one query but an increasingly long T . Interviewers ask it to test tensor dimensions, scaling, masks, complexity, and the difference between mathematical and fused GPU implementations.

2. Core Idea

A query asks, “Which earlier token has features relevant to me?” Each key advertises features. Their dot product is high when aligned.

Small example with $d_h=2$:

q = [2, 1]
k0 = [1, 0] → q·k0 = 2
k1 = [0, 3] → q·k1 = 3

scaled scores = [2/√2, 3/√2]

For all tokens, batched GEMM computes these dot products in parallel. The division by $\sqrt{d_h}$ controls logit magnitude: if vector components have roughly unit variance, an unscaled dot product’s variance grows with d_h , pushing Softmax toward saturation and weak gradients.

Step by step:

- 1. Select one batch item and query head.
- 2. Map it to the correct K head (identical head for MHA; shared head for GQA/MQA).
- 3. Multiply query rows by transposed key rows.
- 4. Scale by $1/\sqrt{d_h}$.
- 5. Add causal, padding, or other mask bias.
- 6. Pass each score row to Softmax—or keep the tile on-chip in a fused attention kernel.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Scaled dot product	Similarity divided by $\sqrt{d_h}$.	$QK^T/\sqrt{128}$.	Prevent overly large logits.
Causal mask	Query at position i may attend only to allowed earlier/current keys.	Set $j>i$ logits to $-\infty$.	Correct triangular direction.

Subtopic	What it means and why it matters	Example	Common interview angle
Padding mask	Excludes padding tokens.	Mask keys past true length.	Mask logits before Softmax.
Complexity	Dense prefill does $O(B \cdot H \cdot S \cdot T \cdot d_h)$ arithmetic.	Self-attention $S=T$ is quadratic in tokens.	Quadratic score matrix vs linear projection.
Score materialization	Naive implementation writes $[B, H, S, T]$ to HBM. Fused kernels tile it on-chip.	FlashAttention-style execution.	Exact same result, lower IO.
Batched/head GEMM	Heads and batches form independent matrix multiplications.	$B \cdot H$ batches of $[S, d_h] @ [d_h, T]$.	Parallel mapping.
GQA head mapping	Multiple Q heads read the same K head.	Q heads 0–3 share K head 0.	Cache traffic/reuse opportunity.
Caches/layout	K may be stored $[B, H_{kv}, T, d_h]$ or paged/interleaved.	Decode gathers cache blocks.	Logical transpose need not mean a physical transpose.

4. Real-World Example

During prefill of a 2048-token causal prompt with 32 heads, a naive implementation conceptually forms $32 \cdot 2048^2 \approx 134$ million score values per sequence. In BF16 that is about 256 MiB if fully materialized, before probabilities or backward state. A fused attention kernel processes score tiles in on-chip memory, applies mask/Softmax, and avoids storing the entire matrix.

During decode, the shape is $[1, d_h] @ [d_h, T] \rightarrow [1, T]$ per query head. Arithmetic is linear in context length for each newly generated token, but reading the KV cache becomes increasingly expensive.

5. Diagrams / Mental Models

```

K positions →   0    1    2    3
Q position 0    [●   ×   ×   ×]
Q position 1    [●   ●   ×   ×]
Q position 2    [●   ●   ●   ×]   causal visibility
Q position 3    [●   ●   ●   ●]

```

Phase	Per-head multiplication	Score shape
Prefill	$[S, d_h] @ [d_h, S]$	$[S, S]$
Decode	$[1, d_h] @ [d_h, T]$	$[1, T]$
Cross-attention	$[S_{dec}, d_h] @ [d_h, S_{enc}]$	$[S_{dec}, S_{enc}]$

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What does QK^T produce?	One compatibility logit per query-key pair.	$[S, T]$ per head.	Calling outputs probabilities.
2. Why transpose K?	Each key vector becomes a column so its dot product with each query is computed.	Shape compatibility.	Saying data must always be physically transposed.
3. Why divide by $\sqrt{d_h}$?	To keep logit variance controlled as head dimension grows.	Softmax stability/saturation.	Dividing by d_h without reason.
4. What is prefill complexity?	Dense self-attention score work is $O(B \cdot H \cdot S^2 \cdot d_h)$.	Quadratic token pairs.	Saying the entire Transformer is only $O(S^2)$ without projection terms.
5. What is decode score complexity?	Per new token it is $O(B \cdot H \cdot T \cdot d_h)$.	One query against all cached keys.	Calling it constant because of KV cache.
6. Where is the mask applied?	To logits before Softmax, commonly via $-\infty$ or a sufficiently negative value.	Zero probability after Softmax.	Masking V instead.
7. Does causal attention skip half the arithmetic?	Semantically half the matrix is invalid; optimized kernels may skip masked tiles, but naive dense GEMM may not.	Math vs implementation.	Assuming mask automatically saves 50%.
8. Why is score materialization costly?	It writes/reads $O(S \cdot T)$ intermediates to/from HBM.	IO bottleneck.	Focusing only on allocation capacity.
9. Is attention always self-attention?	No. Cross-attention has Q from one sequence and K/V from another.	S and T may differ.	Assuming square scores.
10. How does GQA work here?	Each Q head is mapped to a shared KV head; score computation still occurs for every Q head.	Sharing K/V does not eliminate Q heads.	Computing scores for only H_{kv} heads.

7. Deep-Dive Questions

1. **Can FlashAttention change asymptotic FLOPs?** For dense exact attention it primarily reduces HBM IO and materialization; the pairwise arithmetic remains quadratic in sequence length.
2. **What does the causal diagonal mean with a KV cache?** A decode query at the newest position may attend to all cached positions plus itself, so its one row is generally fully valid.

3. **How can a kernel avoid a physical K transpose?** It selects a GEMM/layout interpretation or loads tiles with address calculations that present K in the needed orientation.
4. **Why might long-context decode be bandwidth-bound?** A single query performs limited arithmetic per K element while a growing key cache must be streamed every step.
5. **How can numerical results vary across kernels?** Tiling changes reduction order and accumulation precision; outputs should be close, not necessarily bitwise identical.

8. Comparison Tables

Property	Prefill attention scores	Decode attention scores
Queries per request	S	Usually 1
Keys per request	S	Current context T
Score entries	S^2 per head	T per head
Parallelism	Many query rows	Limited query rows, many heads/batches
Dominant concern	Compute plus score IO	KV-cache bandwidth/latency
Naive attention		IO-aware fused attention
Materializes full score/probability tensors		Processes tiles on-chip
Separate GEMM, mask, Softmax, GEMM kernels		Combines stages in a specialized kernel
More HBM traffic		Less HBM traffic
Easier to understand/debug		More complex implementation, same dense-attention semantics

9. Common Mistakes

- Calling raw scores “attention weights” without clarifying that Softmax is still required.
- Using model width D instead of head width d_h in the scale.
- Getting the causal triangle backward.
- Assuming K^T requires a standalone transpose kernel.
- Claiming KV cache removes attention to old tokens; it removes recomputation of their K/V.
- Confusing quadratic score count with KV-cache size, which is linear in stored tokens.

10. Edge Cases / Special Cases

- All-masked rows need defined handling; naive Softmax of all $-\infty$ yields an undefined $0/0$ form and may produce NaNs.
- Sliding-window attention restricts the visible keys and changes effective T .
- Prefix-LM or block masks are not simply lower triangular.
- Packed batches must prevent attention across unrelated sequences.
- Very long T may require split-K/split-sequence reductions across thread blocks.
- ALiBi or relative-position biases add values to logits rather than rotating Q/K.

11. How to Explain in Interview

For each head, attention scores are $QK^T/\sqrt{d_h}$. This forms one similarity logit per query-key pair, then adds causal or padding masks before Softmax. Prefill has an $S \times S$ score problem, while decode has one query against T cached keys. Efficient kernels tile this computation so the full score matrix never needs to visit HBM.

12. Quick Revision Notes

- Per head: $[S, d_h] @ [d_h, T] \rightarrow [S, T]$.
- Scale by $1/\sqrt{d_h}$, then add masks.
- Scores are logits, not probabilities.
- Prefill pair count is quadratic; decode per-step pair count is linear in context.
- KV cache avoids old K/V projection, not old-key attention.
- Trap: mathematical transpose does not require a separate physical copy.

13. Practice Tasks

1. Hand-compute a 3×3 causal score matrix for $d_h=2$.
2. Derive FLOPs for $B=4, H=32, S=T=2048, d_h=128$.
3. Calculate bytes of a materialized BF16 score tensor.
4. Draw masks for causal, bidirectional, and sliding-window attention.
5. Implement CPU QK^T , scale, and mask; validate forbidden logits.

14. Final Cheat Sheet

Item	Remember
Core definition	Scaled query-key dot products create compatibility logits.
Why it matters	Establishes token relationships and drives attention complexity.
Most asked	Shapes, scale, masks, prefill/decode complexity, FlashAttention.
Main comparison	Prefill is many-query quadratic; decode is one-query linear per step.
One-line answer	"Compute $[S, d_h] \times [d_h, T]$, scale by $\sqrt{d_h}$, mask invalid keys, and feed each row to Softmax."

Softmax in Attention

1. Overview

Definition. Softmax converts each row of attention logits into nonnegative weights that sum to one:

$$p_j = \exp(z_j) / \sum_k \exp(z_k)$$

In attention, each query has one row of T logits, so normalization is across the key dimension—not across heads, batches, or hidden features. Softmax is used to turn arbitrary query-key similarity scores into a weighted distribution over visible values.

It matters on GPUs because it is a reduction, not a plain element-wise operation. Threads must cooperate to find a row maximum and sum exponentials. A naive standalone implementation also reads and writes the large score matrix. Interviewers ask about numerical stability, masks, parallel reduction, and fusion with attention.

2. Core Idea

Softmax is like converting unnormalized preference scores into a budget of one unit. Larger scores receive more of the budget, but all visible positions compete within the same row.

For logits $[1, 2, 3]$, direct exponentials give approximately $[2.72, 7.39, 20.09]$; dividing by their sum gives $[0.09, 0.24, 0.67]$.

Computers use stable Softmax:

$$m = \max(z)$$
$$p_j = \exp(z_j - m) / \sum_k \exp(z_k - m)$$

Subtracting the same constant does not change the result, but guarantees every exponential argument is at most zero. For $[1000, 1001, 1002]$, direct $\exp(1002)$ overflows common floating-point types, while subtracting 1002 yields $[-2, -1, 0]$.

GPU steps for one row:

- 1. Threads load different logits.
- 2. A warp/block reduction computes the maximum.
- 3. Each thread computes $\exp(\text{logit} - \text{max})$; masked entries contribute zero.
- 4. Another reduction computes the sum.
- 5. Each value is divided by the sum.
- 6. Probabilities are written—or immediately multiplied with V inside a fused kernel.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Stable Softmax	Subtract the row maximum before exponentiation.	$[1000, 1001] \rightarrow [-1, 0]$.	Prevent overflow without changing probabilities.
Reduction axis	Normalize over keys for each $(\text{batch}, \text{head}, \text{query})$ row.	Last dimension T .	Correct axis.
Masks	Invalid logits act like $-\infty$, so their exponentials are zero.	Future causal position.	Apply mask before normalization.

Subtopic	What it means and why it matters	Example	Common interview angle
Accumulation precision	Max and sum are often computed in FP32 even when inputs/outputs use FP16/BF16.	BF16 logits, FP32 reduction.	Stability vs storage dtype.
Online Softmax	Maintains a running maximum and rescaled sum across tiles.	Used by tiled attention.	Enables exact Softmax without full row materialization.
Warp/block reductions	Shuffle operations and shared memory combine partial results.	Warp max then block max.	Synchronization and variable row lengths.
Dropout	Training may drop/rescale probabilities; inference normally disables it.	Attention dropout.	Train vs inference semantics.
Log-softmax	Computes log probabilities stably; generally not the attention operation.	Classification loss.	Do not substitute it blindly.

4. Real-World Example

In a fused prefill attention kernel, a thread block loads a tile of Q and K, computes a score tile, and updates per-query running statistics. If a later tile has a larger maximum, earlier accumulated exponentials and the partial **PV** output are rescaled. This produces exact row-wise Softmax while the score tile stays in registers/shared memory.

5. Diagrams / Mental Models

```

logit row → max reduction → subtract max → exp
                                         |
probabilities ← divide by sum ← sum reduction ←

```

Only values in the same query row participate in these reductions.

Stable merge of two Softmax tiles:

```

tile A: max mA, exponential sum lA
tile B: max mB, exponential sum lB
m = max(mA, mB)
l = exp(mA-m) * lA + exp(mB-m) * lB

```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why Softmax?	It turns logits into nonnegative, normalized weights for value aggregation.	Per-query distribution.	Saying it selects exactly one token.
2. Along which axis?	Across keys T for each batch/head/query row.	Correct row semantics.	Normalizing across heads.
3. Why subtract the maximum?	It prevents positive exponential overflow and leaves Softmax unchanged.	Shift invariance.	Subtracting the mean.
4. How are masked entries handled?	Give them $-\infty$ logically, making their exponential contribution zero.	Mask before Softmax.	Zeroing logits; $\exp(0)=1$.
5. Why is Softmax a reduction?	Every output denominator depends on the sum of all row exponentials, and stability needs a row maximum.	Two collective stages.	Calling it purely element-wise.
6. Why may FP32 be used internally?	Wider range/precision makes max-sum normalization more stable.	Mixed precision.	Assuming output must also be FP32.
7. What is online Softmax?	An exact running max/sum method that merges tiles with rescaling.	FlashAttention connection.	Calling it approximate.
8. Is Softmax always a separate kernel?	No; optimized attention commonly fuses it with score and V aggregation.	Graph vs implementation.	Counting framework ops as kernels.
9. What happens if all entries are masked?	Ordinary formula has zero denominator and needs explicit handling/defined semantics.	NaN edge case.	Assuming it naturally returns zeros.
10. Why can long rows be hard?	They require cross-warp/block reduction and more data movement; one block may not cover the row conveniently.	Parallel reduction strategy.	Only mentioning exponential cost.

7. Deep-Dive Questions

- 1. Prove shift invariance.** $\exp(z_j - c) / \sum \exp(z_k - c)$ has common factor $\exp(-c)$ in numerator and denominator, which cancels.
- 2. How can online Softmax revise an already accumulated output?** When the running max increases, multiply the old denominator and weighted-value accumulator by $\exp(\text{old_max} - \text{new_max})$ before merging the new tile.
- 3. Why is a finite large-negative mask sometimes risky?** If it is not sufficiently negative for the dtype/scale, masked entries can retain nonzero probability; extreme constants can also interact poorly with arithmetic.

4. **Can Softmax be computed independently per tile without correction?** No. Every tile needs globally consistent normalization; online merging supplies the necessary max and denominator correction.
5. **Why are outputs sometimes not bitwise identical across kernels?** Reduction order, approximate exponential instructions, and intermediate precision differ.

8. Comparison Tables

Stable Softmax	Naive Softmax
Subtracts row maximum	Exponentiates raw logits
Avoids positive overflow	Can overflow for large logits
Same mathematical result	Same only when arithmetic stays representable
Production standard	Teaching-only formula
Standalone Softmax	Fused/online Softmax
Reads/writes score matrix in HBM	Keeps score tiles and state on-chip
Simple kernel boundaries	More complex tiling/rescaling
Easy to inspect probabilities	Avoids large intermediate traffic
Useful fallback/debug path	Preferred for efficient attention when supported

9. Common Mistakes

- Normalizing over the head or query dimension.
- Setting masked logits to zero instead of negative infinity.
- Omitting max subtraction.
- Believing Softmax makes the largest logit exactly one.
- Saying online Softmax approximates the denominator.
- Forgetting inference disables attention dropout.
- Treating exp latency as the only cost and ignoring score-tensor IO.

10. Edge Cases / Special Cases

- A single valid logit produces probability one.
- Equal logits over n valid positions produce $1/n$ each.
- $+\infty$, $-\infty$, or NaN inputs need well-defined framework behavior; NaNs usually propagate.
- All-masked rows need a guard or higher-level guarantee that they cannot occur.
- Large T can require split reductions and an extra merge stage.
- Quantized attention normally dequantizes or uses wider accumulators before Softmax.

11. How to Explain in Interview

Attention Softmax operates row-wise over keys. A stable implementation subtracts each row's maximum, exponentiates, reduces the sum, and normalizes. On GPUs those max and sum operations are parallel reductions. Fused attention uses online Softmax so score tiles never need to be written to HBM.

12. Quick Revision Notes

- Normalize across visible keys for each query/head.
- Stable form: $\exp(z - \max(z)) / \sum(\exp(z - \max(z)))$.
- Mask with logical $-\infty$ before Softmax.
- Two reductions: max and sum.
- Online Softmax is exact and tile-mergeable.
- FP32 accumulation is common with low-precision storage.
- Trap: zero logit is not zero probability.

13. Practice Tasks

1. Compute stable Softmax by hand for $[1000, 1001, 1002]$.
2. Implement CPU stable Softmax and assert every row sums approximately to one.
3. Add a causal mask and verify masked probabilities are zero.
4. Derive the two-tile online merge formula.
5. Sketch a warp-level max reduction with shuffle operations.

14. Final Cheat Sheet

Item	Remember
Core definition	Row-wise normalization of attention logits into weights.
Why it matters	Determines value mixing and requires numerically stable reductions.
Most asked	Axis, max subtraction, masks, reductions, online Softmax.
Main comparison	Standalone materializes scores; fused online Softmax keeps tiles on-chip.
One-line answer	"Subtract the row max, exponentiate valid logits, sum them, and normalize across keys."

Attention $\times$ V (PV)

1. Overview

Definition. After Softmax, the probability matrix P multiplies the value matrix:

$$O_{\text{head}} = P V$$

Per head, $P: [S, T]$ and $V: [T, d_h]$ produce $O_{\text{head}}: [S, d_h]$. Each query output is a weighted sum of value vectors from visible positions.

This is the point where selected information is actually gathered. QK^T determines relevance; PV mixes content. It matters on GPUs because it is the second attention matrix multiplication and because efficient kernels

combine it with online Softmax to avoid materializing P . Interviewers ask about shapes, semantics, complexity, and fused attention.

2. Core Idea

Imagine three source value vectors and probabilities $[0.2, 0.7, 0.1]$. The output is a blend:

```
V0 = [1,0]      0.2·V0 = [0.2,0.0]
V1 = [0,2]      0.7·V1 = [0.0,1.4]
V2 = [3,1]      0.1·V2 = [0.3,0.1]
output                                [0.5,1.5]
```

Step by step:

1. Take one query's normalized attention row of length T .
2. Multiply each scalar probability by its associated d_h -wide value vector.
3. Sum across T .
4. Repeat for every query, head, and batch element.
5. Concatenate all query-head outputs into width D for the output projection.

In a naive pipeline, P is written after Softmax and read again by a GEMM. In a fused kernel, a score tile is normalized and immediately used to update an output accumulator while its corresponding V tile is on-chip.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Weighted sum	Each output feature is a reduction over key positions.	$O[i,c] = \sum_j P[i,j]V[j,c]$.	Explain what attention retrieves.
GEMM shape	$[S,T]@[T,d_h] \rightarrow [S,d_h]$ per head.	Reduction axis is T .	Dimension reasoning.
Fused accumulation	Online Softmax state and partial PV output are updated together.	FlashAttention.	Why no full P tensor is needed.
Value-cache access	Decode reads historical V for every active sequence.	$[T,d_h]$ per KV head.	Bandwidth grows with context.
GQA/MQA sharing	Multiple query heads may read the same V head but use different probabilities.	Four Q heads share V head.	V shared; outputs remain per- Q -head.
Accumulator precision	Weighted sums may accumulate in FP32.	Low-precision P/V , wider accumulator.	Accuracy vs registers.

Subtopic	What it means and why it matters	Example	Common interview angle
Head concatenation	Per-head outputs are rearranged/viewed as $[B, S, H \cdot d_h]$.	$[B, H, S, d_h] \rightarrow [B, S, D]$.	Layout transpose may cost.

4. Real-World Example

For a decode step with 32 query heads, $T=8192$, and $d_h=128$, each head combines 8192 cached value vectors into one 128-value output. Even though only one token is generated, every relevant historical V entry may be read. GQA reduces the number of distinct V heads stored, enabling query heads in a group to reuse the same cache region, though each still has its own probability row and output.

5. Diagrams / Mental Models

```

probabilities for query i
[p0 p1 p2 ... pT-1]
  | scalar weights
  ▼
V = [ value vector 0 ]
    [ value vector 1 ] → weighted vector sum → O_head[i,:]
    [ value vector 2 ]
    [      ...      ]

```

Data	Naive location between stages	Fused location
Score tile	HBM as full matrix	Registers/shared memory
Probability tile	HBM as full matrix	Registers/shared memory
Output accumulator	GEMM registers	Registers, rescaled online

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What does PV compute?	A probability-weighted sum of value vectors for each query/head.	Retrieval/content mixing.	Saying it computes similarity.
2. What are its shapes?	$[S, T] @ [T, d_h] \rightarrow [S, d_h]$ per head.	Correct reduction axis.	Using $[T, S]$.
3. Do probabilities mix features?	Each scalar weights an entire V row; the sum is over token positions, independently for each value feature.	Token-axis reduction.	Softmax over features.

Question	Clear answer	Interviewer expects	Common mistake
4. Why use V instead of K?	K is optimized for matching; V carries the content to aggregate.	Learned role separation.	Claiming K and V must be identical.
5. What is the prefill complexity?	$O(B \cdot H \cdot S \cdot T \cdot d_h)$, quadratic for dense self-attention where $S=T$.	Same order as QK^T .	Counting only probability entries.
6. What is decode complexity?	$O(B \cdot H \cdot T \cdot d_h)$ per generated token.	Linear in context per step.	Calling it $O(1)$ with cache.
7. Why fuse it with Softmax?	Avoid writing and rereading P, and reuse V/score tiles on-chip.	IO reduction.	Saying fusion changes probabilities.
8. How does GQA affect PV?	Q heads retain separate P/output but groups reference the same V head.	Shared cache, distinct attention.	Producing only H_{kv} outputs.
9. Where are heads combined?	After per-head PV, outputs are rearranged/concatenated along feature width.	$[H, d_h] \rightarrow D$.	Summing heads.
10. Can output be computed before seeing all tiles?	Partially, but online Softmax must rescale prior partial output if a later tile changes the row max/denominator.	Correct online algorithm.	Adding independently normalized tile outputs.

7. Deep-Dive Questions

- Why can a fused kernel maintain an unnormalized numerator?** It tracks $\sum \exp(\text{score}-m) \cdot V$ and denominator $\sum \exp(\text{score}-m)$ under the same running maximum, then divides at the end.
- Why does decode not achieve large GEMM efficiency here?** The query dimension is one, so there are fewer output rows and less reuse than a large prefill GEMM.
- What is the effect of a nearly one-hot Softmax?** The result approaches the value vector at the dominant position, though the implementation still usually processes the row.
- Can zero probability entries be skipped?** Exact dense Softmax rarely creates explicit structural sparsity; specialized sparse attention can skip known masked blocks.
- How does value quantization affect execution?** It reduces cache bytes but requires scale handling/dequantization and can perturb the weighted sum; kernel support determines net benefit.

8. Comparison Tables

Operation	QK^T	PV
Purpose	Compute relevance logits	Aggregate content
Inputs per head	$[S, d_h], [T, d_h]$	$[S, T], [T, d_h]$
Output	$[S, T]$ scores	$[S, d_h]$ head output
Reduction dimension	d_h	T

Operation	QK^T	PV
Main large intermediate	Creates scores	Consumes probabilities
Prefill PV	Decode PV	
Many query rows	One query row per active request	
Large matrix multiply	Matrix-vector-like per head	
High parallelism	KV-cache streaming dominates at long context	

9. Common Mistakes

- Saying PV creates attention probabilities.
- Summing head outputs instead of concatenating them.
- Forgetting that GQA shares V inputs, not query-head outputs.
- Assuming KV cache makes the per-step PV independent of context length.
- Independently normalizing score tiles and then adding their outputs.
- Ignoring accumulator precision and layout conversion.

10. Edge Cases / Special Cases

- With one valid key, the output equals that key's V vector.
- With uniform probabilities, the output is the mean of visible V rows.
- Empty/all-masked key sets require explicit behavior.
- Sliding-window/sparse attention reads only the allowed V subset.
- Paged KV caches require a block table to locate logical positions.
- Split-sequence decode kernels need a second reduction to merge partial outputs with correct Softmax statistics.

11. How to Explain in Interview

PV takes each query's Softmax row and forms a weighted sum of value vectors, producing one d_h -wide result per head. Its shape is $[S, T] \times [T, d_h] \rightarrow [S, d_h]$. Fused attention performs this accumulation tile by tile with online Softmax, avoiding a full probability matrix in HBM.

12. Quick Revision Notes

- Formula: $O[i, c] = \sum_j P[i, j] V[j, c]$.
- Per-head shape: $[S, T] @ [T, d_h] \rightarrow [S, d_h]$.
- QK selects; V carries content.
- Heads are concatenated, not averaged.
- Decode reads V cache up to context T.
- Online output must be rescaled when running Softmax max changes.
- Trap: KV cache reduces recomputation, not current attention scanning.

13. Practice Tasks

1. Hand-calculate a weighted sum for three 2D values.

- 2. Implement CPU **PV** and validate output shape.
- 3. Estimate V-cache bytes read per decode step for chosen model dimensions.
- 4. Draw MHA and GQA V-head mappings.
- 5. Explain why independently normalized tile results cannot simply be averaged.

14. Final Cheat Sheet

Item	Remember
Core definition	Probability-weighted sum of value vectors.
Why it matters	Produces the information retrieved by attention.
Most asked	Shape, semantic role, complexity, fusion, GQA.
Main comparison	QK^T finds relevance; PV gathers content.
One-line answer	"Multiply each Softmax row by V to reduce over context and produce one vector per query head."

Output Projection

1. Overview

Definition. The output projection combines the concatenated head outputs through a learned matrix:

```
O_concat = concat(O_0, O_1, ..., O_H-1)    # [B,S,D]
Y = O_concat W_O + b_O                    # W_O: [D,D]
```

Attention heads operate in separate subspaces. Concatenation restores the model-width vector, and **W_O** learns how features from different heads should interact before the result rejoins the residual stream.

It matters because it is another large GEMM, because its tensor-parallel communication pattern differs from QKV projection, and because its epilogue may fuse bias and residual addition. Interviewers ask it to ensure candidates do not stop the attention explanation at **PV**.

2. Core Idea

Think of heads as specialist teams. Each returns a report of width **d_h**. Concatenation places the reports side by side; the output projection is the editor that mixes their features into one **D**-wide update.

For **H=2, d_h=2, D=4**:

```
head 0 output = [a,b]
head 1 output = [c,d]
```

```
concat      = [a,b,c,d]
[a,b,c,d] @ W_0[4,4] → [y0,y1,y2,y3]
```

GPU execution:

1. Attention produces per-head output, logically $[B, H, S, d_h]$.
2. A view, transpose, copy, or fused write creates the $[B, S, D]$ layout expected by GEMM.
3. Flatten to $[B \cdot S, D]$.
4. Execute $[B \cdot S, D] @ [D, D]$.
5. Apply optional bias and add the residual, often in a GEMM epilogue or nearby fused kernel.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Head concatenation	Join feature axes, preserving every head's output.	$H \cdot d_h = D$.	Concatenation vs sum.
Output GEMM	Learned mixing across all head features.	$[B \cdot S, D] @ [D, D]$.	Parameter/FLOP count.
Layout transformation	Attention-friendly $[B, H, S, d_h]$ may not match GEMM-friendly $[B, S, D]$.	transpose head/token axes.	View vs physical copy.
Epilogue fusion	Bias and residual addition can be performed while output tiles are written.	$Y = \text{GEMM} + \text{bias} + \text{residual}$.	Reduced memory traffic.
Row-parallel sharding	When each rank owns input head slices, each computes a partial output and all-reduces/reduce-scatters.	Megatron-style row parallel.	Communication placement.
Quantized weights	Lower bytes for W_0 ; accumulate into wider output.	INT8/FP8/INT4.	Bandwidth vs accuracy/kernel support.
Residual semantics	Projection produces an update, normally added to block input.	$x + W_0 \cdot \text{attention}$.	Full block dataflow.

4. Real-World Example

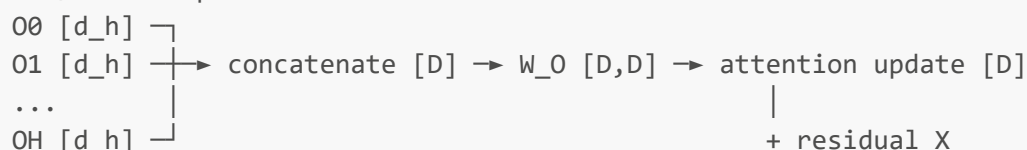
With $D=4096$, the output matrix has about 16.8 million weights. In BF16 it occupies about 32 MiB for one layer. During batch-one decode, those weights must be streamed to produce only one token row, so reuse is poor. With a larger batch, the same weight tile serves more rows and arithmetic intensity improves.

Under tensor parallelism, each GPU may own several attention heads and the matching rows of W_0 . It computes a partial $[B \cdot S, D]$ result. An all-reduce sums partial results so every rank obtains the complete

residual-stream update.

5. Diagrams / Mental Models

GPU/head outputs



Stage	Logical token mixing?	Hidden-feature mixing?
Head concatenation	No	No; layout/join only
Output projection	No	Yes, across head features
Residual addition	No	Element-wise combination

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why is output projection needed?	It learns to mix information from concatenated heads into the model-width residual stream.	Cross-head feature mixing.	Saying it computes attention scores.
2. What is its shape?	Usually $[B \cdot S, D] @ [D, D] \rightarrow [B \cdot S, D]$.	Correct GEMM dimensions.	Using $[H, H]$.
3. Are heads added or concatenated?	Standard multi-head attention concatenates then projects.	Feature-axis join.	Averaging heads.
4. Does W_O mix token positions?	No, it applies independently to every token row.	Feature mixing only.	Confusing matrix columns with tokens.
5. What happens after projection?	Usually bias if present, residual addition, and then the model's normalization/block ordering.	Full dataflow.	Omitting residual.
6. Why can decode be bandwidth-bound?	A large $D \times D$ matrix is used for only B rows, so weight reuse is limited.	Arithmetic intensity.	Saying the output tensor is too large.
7. Can head concatenation be free?	It may be a view or produced directly in desired layout, but an incompatible physical layout requires a transpose/copy.	Layout-dependent cost.	Always free or always a kernel.

Question	Clear answer	Interviewer expects	Common mistake
8. How is it tensor-parallelized?	Common row-parallel scheme consumes sharded head features, creates partial full-width outputs, then reduces across ranks.	All-reduce/reduce-scatter.	Gathering heads then redundantly multiplying full weights.
9. What can fuse with it?	Bias, residual addition, sometimes quantization or neighboring element-wise epilogue work.	GEMM epilogue.	Assuming arbitrary attention can fuse into the same GEMM.
10. How many parameters?	D^2 plus optional D bias for the common square projection.	Parameter accounting.	Multiplying by number of heads again.

7. Deep-Dive Questions

- Why pair column-parallel QKV with row-parallel output projection?** QKV shards independent output heads without communication; W_O consumes those shards and one reduction reconstructs the full result.
- Can all-reduce be delayed?** Sometimes residual/parallel branch structures allow communication overlap or reduce-scatter variants, but dependent full-width operations constrain delay.
- Why might a fused attention kernel choose its output layout carefully?** Writing directly in the projection's preferred layout avoids a standalone transpose.
- Does W_O restore token interaction lost by per-head processing?** Token interaction already occurred within each head's attention; W_O mixes feature channels across heads for each token.
- How does batch size change this GEMM's roofline position?** Larger $M=B \cdot S$ reuses weight tiles across more rows, increasing FLOPs per weight byte and moving toward compute-bound execution.

8. Comparison Tables

QKV projection	Output projection
Expands/splits into Q, K, V/head features	Mixes concatenated head output back into D
Often column-parallel	Often row-parallel
Can proceed with local head shards	Produces partial outputs requiring reduction
Before token interaction	After value aggregation
Concatenation	Projection
Rearranges/joins features	Learned multiplication
No parameters/FLOPs beyond movement	D^2 parameters and GEMM FLOPs
May be metadata-only	Always performs arithmetic

9. Common Mistakes

- Treating concatenation itself as a learned operation.
- Assuming the heads are summed.
- Forgetting W_O and undercounting attention parameters/FLOPs.
- Saying output projection mixes sequence positions.
- Ignoring the tensor-parallel reduction.
- Assuming a transpose in notation always causes a copy.

10. Edge Cases / Special Cases

- Some architectures omit projection bias.
- $H \cdot d_h$ may differ from D in variants; then W_O is rectangular.
- Head pruning changes input width and requires matching weights/layout.
- Tensor-parallel collectives may be hidden inside a fused communication kernel but remain a data dependency.
- Small decode batches may be dominated by launch and weight movement.
- Quantized output projection may need residual values in a compatible scale/type.

11. How to Explain in Interview

After PV, every query has one vector per head. Standard attention concatenates these vectors into width D and applies W_O to mix features across heads. The projection is token-wise, usually followed by a residual addition, and in tensor parallelism it is commonly row-sharded with an all-reduce of partial outputs.

12. Quick Revision Notes

- `concat(heads)` gives $[B, S, H \cdot d_h]$, commonly $[B, S, D]$.
- Common GEMM: $[B \cdot S, D] @ [D, D]$.
- Projection mixes features/heads, not tokens.
- Bias + residual are good epilogue-fusion candidates.
- Decode has weak weight reuse at small batch.
- TP pattern: local partial projection, then reduction.
- Trap: concatenation is not summation.

13. Practice Tasks

1. Derive output projection shape and FLOPs for $B=2, S=1024, D=4096$.
2. Calculate W_O memory in FP32, BF16, INT8, and INT4.
3. Draw column-parallel QKV followed by row-parallel W_O on two GPUs.
4. Test whether a framework's head transpose is a view or creates a contiguous copy.
5. Compare arithmetic intensity for decode batch 1 and batch 32 conceptually.

14. Final Cheat Sheet

Item	Remember
Core definition	Learned mixing of concatenated per-head results.
Why it matters	Returns attention output to model width; major GEMM/TP boundary.

Item	Remember
Most asked	Shape, concatenation, residual, bandwidth, TP reduction.
Main comparison	QKV fans out into heads; W_O combines head features.
One-line answer	"Concatenate the head results, multiply by W_O , and add the residual; under TP the partial projections are reduced."

Transformer MLP

1. Overview

Definition. The MLP/FFN is a token-wise nonlinear sublayer. A classic form is:

$$\text{MLP}(x) = \text{activation}(xW_{\text{up}} + b_{\text{up}}) W_{\text{down}} + b_{\text{down}}$$

Many modern LLMs use a gated form such as SwiGLU:

```
u = xW_up
g = xW_gate
h = SiLU(g) ⊙ u
y = hW_down
```

The MLP does not mix tokens; it expands each token from width D to D_{ff} , applies a nonlinearity/gate, and projects back. It often contains more parameters and projection FLOPs than attention. Interviewers ask it to test GEMM accounting, activation fusion, tensor parallelism, and the attention-vs-MLP distinction.

2. Core Idea

Attention lets a token gather information from other positions. The MLP then transforms the gathered features independently for that token. Think of attention as communication among employees and the MLP as each employee privately processing what they heard.

Classic example for one token:

```
x [D]
  | w_up [D,D_ff]
  ▼
wide vector [D_ff]
  | GELU/ReLU
  ▼
activated wide vector [D_ff]
  | w_down [D_ff,D]
```

▼
update [D] + residual

GPU steps for SwiGLU:

1. Normalize the $[B \cdot S, D]$ input, depending on block architecture.
2. Compute **up** and **gate**, often with one packed GEMM of output width $2D_{ff}$.
3. Apply $\text{SiLU}(\text{gate}) * \text{up}$, ideally without a separate HBM round trip.
4. Run the down-projection GEMM $[B \cdot S, D_{ff}] @ [D_{ff}, D]$.
5. Add the residual, often through an epilogue/fused element-wise kernel.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Expansion ratio	D_{ff} is wider than D , creating feature capacity.	$D=4096, D_{ff}=11008$.	Shapes/parameter count.
Activation	Nonlinearity makes stacked linear layers more expressive than one linear map.	GELU, SiLU.	Without activation, matrices collapse algebraically.
Gating	One branch controls another element-wise.	$\text{SiLU}(g) \odot u$.	SwiGLU has two input projections.
Packed gate/up	Compute both branches with one GEMM.	$[M, D] @ [D, 2D_{ff}]$.	Launch/input reuse.
Activation fusion	Apply bias/activation/gating as tiles are produced or before down GEMM.	fused SwiGLU.	Avoid large intermediate traffic.
Tensor parallelism	Gate/up commonly column-sharded; down projection row-sharded and reduced.	Megatron MLP.	Same paired pattern as QKV/ w_o .
Quantization	MLP weights are a large fraction of model bytes.	INT8/INT4 weight-only.	Decode bandwidth savings.
Mixture of Experts	Routes tokens to selected expert MLPs.	top-2 experts.	Adds routing/all-to-all/load-balance concerns; not ordinary dense MLP.

4. Real-World Example

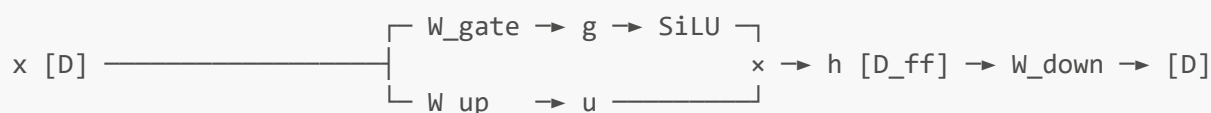
For $D=4096, D_{ff}=11008$, a bias-free SwiGLU layer has two input matrices plus one down matrix:

$$\text{parameters} = 2 \cdot D \cdot D_{ff} + D_{ff} \cdot D = 3 \cdot 4096 \cdot 11008$$

≈ 135.3 million weights per layer

At BF16, that is about 258 MiB of MLP weights per layer. For batch-one decode, every token step streams these large weights with little reuse. This is why weight quantization and batching strongly affect LLM decode throughput.

5. Diagrams / Mental Models



token 0 → same MLP weights → token 0 output

token 1 → same MLP weights → token 1 output (no cross-token edges)

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What does the MLP do?	It independently transforms each token through expansion, nonlinearity/gating, and contraction.	Token-wise feature processing.	Saying it attends to other tokens.
2. What are classic FFN shapes?	$[M, D] @ [D, D_{ff}] \rightarrow [M, D_{ff}]$, then $[M, D_{ff}] @ [D_{ff}, D] \rightarrow [M, D]$.	$M = B \cdot S$.	Reducing over sequence length.
3. Why is activation necessary?	Without it, two linear maps compose into a single linear map.	Nonlinear expressiveness.	"It only prevents vanishing gradients."
4. What is SwiGLU?	$\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}})$ followed by down projection.	Two branches plus gating.	Applying SiLU after the element-wise product.
5. Does the MLP mix tokens?	No; each token row is processed independently with shared weights.	Attention vs MLP.	Confusing batch GEMM with token mixing.
6. Why fuse gate and up projections?	One packed GEMM reduces launches and can reuse input loading.	Same parameters/math.	Saying the branches become identical.
7. Why is MLP important for runtime?	Its large matrices often dominate parameters and projection FLOPs/weight traffic.	Model-dependent but substantial cost.	Assuming attention always dominates.

Question	Clear answer	Interviewer expects	Common mistake
8. How is it tensor-parallelized?	Gate/up output features are sharded; down projection consumes those shards and reductions combine partial outputs.	Column then row parallel.	All-reduce after every GEMM.
9. What can fuse?	Bias, activation, gate multiply, residual, and quantization steps depending on kernel boundaries.	Intermediate traffic reduction.	Expecting both large GEMMs to become one algebraic GEMM despite nonlinearity.
10. How does decode affect MLP efficiency?	$M=B$ is small, so weights have little reuse and bandwidth/launch overhead can dominate.	Batch improves reuse.	Claiming sequence length directly changes one step's MLP shape beyond batch.

7. Deep-Dive Questions

- Why can two linear layers be collapsed only without activation/gating?** Matrix multiplication is associative: $(xW_1)W_2 = x(W_1W_2)$. A nonlinear element-wise function breaks that equivalence.
- Why is D_{ff} for SwiGLU often not exactly $4D$?** Gated MLPs use three weight matrices instead of two; architectures choose widths to balance parameter/FLOP budgets and hardware alignment.
- Can activation be fused into the down GEMM?** A specialized kernel can consume generated tiles without a global intermediate, but standard library GEMM boundaries and resource constraints may limit full fusion.
- What makes MoE execution harder?** Tokens are dynamically routed, causing irregular expert batch sizes, load imbalance, data movement/all-to-all, and many smaller GEMMs.
- Why may a quantized MLP still not reach the nominal bit-width speedup?** Scale loads, dequantization, unsupported Tensor Core paths, other activation traffic, and non-GEMM overhead remain.

8. Comparison Tables

Property	Attention	Dense MLP
Mixes token positions	Yes	No
Main axes	Sequence pairs and head features	Hidden/intermediate features
Context-length sensitivity per decode step	KV reads grow with T	Shape mainly depends on active token count B
Main weights	QKV and output projections	Gate/up/down projections

Property	Attention	Dense MLP
Common optimization	Fused tiled attention, KV cache	Packed projections, activation fusion, quantization
Classic FFN	SwiGLU-style MLP	
One up projection	Gate and up projections	
GELU/ReLU commonly	SiLU gate commonly	
Two main weight matrices	Three main weight matrices	
$\text{act}(\mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$	$(\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot \mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$	

9. Common Mistakes

- Saying the MLP communicates between tokens.
- Forgetting the second SwiGLU input projection in parameter/FLOP counts.
- Assuming $D_{\text{ff}}=4D$ for every architecture.
- Thinking two GEMMs can always collapse despite nonlinearity.
- Ignoring residual and normalization around the MLP.
- Treating MoE as a drop-in dense GEMM with identical execution behavior.

10. Edge Cases / Special Cases

- Bias may be absent in many LLM architectures.
- Intermediate widths are often rounded for Tensor Core/alignment efficiency.
- Very small token counts may use GEMV-specialized or grouped kernels.
- Activation overflow/underflow and quantization scales require wider intermediate handling.
- MoE capacity limits may drop, reroute, or pad tokens during training/serving.
- Some architectures run attention and MLP branches in parallel from the same normalized input.

11. How to Explain in Interview

The Transformer MLP is a per-token feature transformation: expand from D to D_{ff} , apply a nonlinearity or gate, then project back to D and add the residual. A SwiGLU MLP uses gate and up projections, multiplies $\text{SiLU}(\text{gate})$ by up , and then applies the down projection. Its large weights make it a major decode bandwidth cost.

12. Quick Revision Notes

- MLP is token-wise; attention is the token mixer.
- Classic: $\text{act}(\mathbf{X}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$.
- SwiGLU: $(\text{SiLU}(\mathbf{X}\mathbf{W}_{\text{gate}}) \odot \mathbf{X}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$.
- Shapes use $M=B \cdot S$, D , and D_{ff} .
- Gate/up can be packed; activation/gate/residual can be fused.
- TP: column-shard expansion, row-shard contraction, reduce afterward.
- Trap: D_{ff} is architecture-specific.

13. Practice Tasks

- 1. Calculate classic and SwiGLU parameter counts for chosen D, D_{ff} .
- 2. Implement a small CPU SwiGLU and validate dimensions.
- 3. Explain why removing SiLU changes representational power.
- 4. Compare BF16 and INT4 MLP weight bytes.
- 5. Draw a two-GPU column/row-parallel MLP.

14. Final Cheat Sheet

Item	Remember
Core definition	Per-token expand, nonlinear/gated transform, and contract.
Why it matters	Large share of Transformer parameters, FLOPs, and decode weight traffic.
Most asked	Shapes, SwiGLU, token independence, fusion, TP.
Main comparison	Attention mixes tokens; MLP transforms each token's features.
One-line answer	"The MLP independently expands each token, applies a nonlinear gate, projects back to model width, and adds the residual."

Kernel Fusion

1. Overview

Definition. Kernel fusion combines operations that would otherwise launch as separate GPU kernels into one kernel that produces the same logical result.

For example:

```
separate: GEMM → write → bias kernel → write → GELU kernel → write
fused:    GEMM with bias+GELU epilogue → one final write
```

Fusion matters because intermediate tensors otherwise travel through HBM and every kernel has launch/scheduling overhead. Transformer inference contains many element-wise operations and reductions around large GEMMs, so fusion is central to production runtimes, compilers, and hand-written CUDA kernels. Interviewers ask it to see whether candidates understand that fewer framework operations, fewer kernels, and fewer FLOPs are different things.

2. Core Idea

Imagine cooking a meal. A non-fused workflow puts an ingredient back in the refrigerator after every tiny step, then retrieves it again. Fusion keeps it on the counter—or in your hand—until the sequence of operations is complete.

GPU equivalent:

- HBM is large but comparatively expensive to revisit.
- Registers/shared memory are small and fast.
- A kernel launch has fixed CPU/driver scheduling cost.
- If a producer and consumer can share the same tiling/thread mapping, an intermediate can stay on-chip.

Small example for $y = \text{ReLU}(a*x+b)$ over N elements:

Three kernels might read/write:

```
1. t1 = a*x      read x, write t1
2. t2 = t1+b     read t1, write t2
3. y  = ReLU(t2) read t2, write y
```

One fused kernel:

```
y[i] = max(0, a*x[i]+b)  read x once, write y once
```

The arithmetic is almost unchanged. The saved work is intermediate memory traffic and launches.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Vertical fusion	Fuse producer and consumer operations.	bias + activation after GEMM.	Eliminate intermediate HBM traffic.
Horizontal fusion	Combine independent small operations/kernels into one launch.	process Q/K/V transforms together.	Improve launch amortization.
GEMM epilogue	Apply element-wise work as accumulator tiles are stored.	bias, residual, activation.	Natural high-value fusion point.
Attention fusion	Tile QK, mask, Softmax, and PV together.	FlashAttention.	Avoid $S \times T$ intermediates.
Persistent kernel	Keep work/state resident and loop over tasks to reduce relaunch/data movement.	specialized decode kernels.	Resource and scheduling trade-offs.
Compiler fusion	Graph/JIT compiler recognizes compatible operations.	XLA, TorchInductor, TensorRT.	Dynamic shapes or aliases can block fusion.
Register pressure	More fused live values consume registers, possibly causing spills/lower occupancy.	fused norm+projection.	Fusion can become slower.
Synchronization scope	Operations requiring global synchronization often need a kernel	multi-block global reduction.	Not everything can fuse safely.

Subtopic	What it means and why it matters	Example	Common interview angle
	boundary unless special cooperative mechanisms apply.		

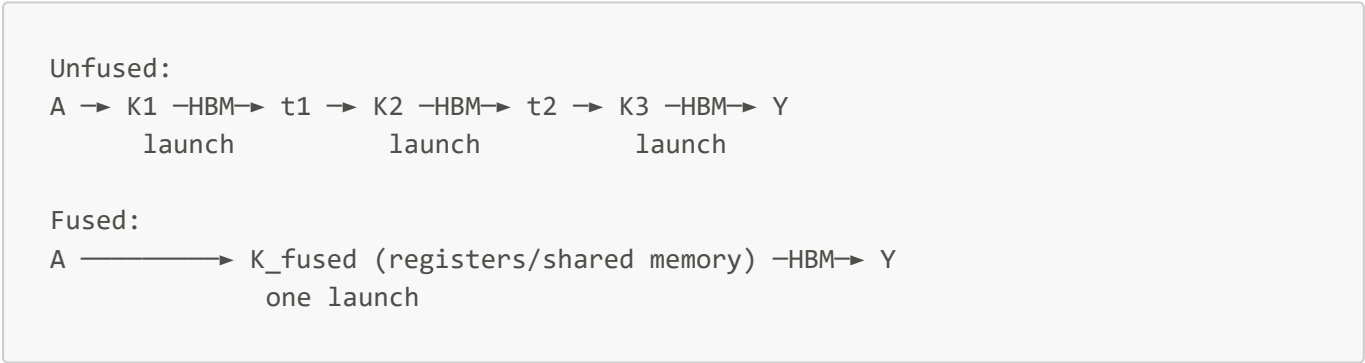
4. Real-World Example

A Transformer layer might logically contain RMSNorm, QKV projection, bias, RoPE, KV-cache write, attention, output projection, residual addition, a second RMSNorm, gate/up projection, SwiGLU, down projection, and another residual addition. A production engine does not necessarily launch one kernel per box. It may use:

- fused RMSNorm and residual handling;
- one packed QKV GEMM with a fused epilogue;
- a fused attention kernel for QK/mask/Softmax/PV;
- a fused residual/bias epilogue on output projection;
- packed gate/up GEMM and fused SwiGLU;
- fused down-projection epilogue.

The exact boundaries depend on shapes, dtype, hardware, and supported kernels.

5. Diagrams / Mental Models



Fusion benefit	When strongest
Save launch overhead	Many tiny/short kernels, especially decode
Save HBM traffic	Large intermediate tensors
Improve locality	Producer/consumer share compatible tiles
Enable recomputation trade	Cheap values can be recomputed instead of stored

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is kernel fusion?	Executing multiple logical operations in one GPU kernel.	Same semantics, different implementation boundary.	Saying it merges CUDA source files.

Question	Clear answer	Interviewer expects	Common mistake
2. Why is it faster?	It can remove launches and intermediate HBM reads/writes, and improve locality.	Concrete bottlenecks.	Saying it always reduces FLOPs.
3. Give a Transformer example.	Fused bias+activation, RMSNorm+residual, or QK+mask+Softmax+PV.	Relevant operation chain.	Only mentioning convolution.
4. Does fusion always help?	No. Register/shared-memory pressure, occupancy loss, code complexity, or incompatible mappings can outweigh savings.	Trade-offs.	"One giant kernel is always best."
5. What is an epilogue?	Work performed on GEMM accumulator results before/while final storage.	Bias/activation/residual.	Confusing it with a separate kernel.
6. How does FlashAttention use fusion?	It tiles score computation, masking, online Softmax, and V accumulation so full scores/probabilities are not stored.	IO awareness.	Claiming it approximates attention.
7. What blocks fusion?	Global synchronization, incompatible layouts/tiling, aliases, unsupported dynamic behavior, resource limits.	Correctness plus resources.	Only compiler limitations.
8. How do you verify fusion?	Inspect profiler kernel timeline/generated graph and measure end-to-end latency/traffic.	Evidence, not source-code assumption.	Counting high-level operators.
9. Why is fusion valuable in decode?	Individual operations are small, so launch overhead and repeated weight/activation traffic matter more.	Phase-specific reasoning.	Assuming only prefill benefits.
10. Is CUDA Graph capture fusion?	No. Graphs reduce launch overhead but kernels remain distinct; both techniques can coexist.	Distinguish scheduling from fusion.	Calling a graph one fused kernel.

7. Deep-Dive Questions

1. **Why can full MLP fusion be difficult?** The intermediate has a nonlinear transform between two large GEMMs; combining their tiling while keeping enough data on-chip can exceed register/shared-memory capacity.
2. **How can fusion lower occupancy?** More live values and larger shared-memory tiles increase per-block resources, reducing resident blocks/warps.

3. **Why can recomputation be a fusion technique?** Recomputing a cheap scalar such as an index, mask, or normalization factor can cost fewer cycles than storing and reloading it from HBM.
4. **What is the difference between operator fusion and kernel specialization?** Fusion combines operations; specialization chooses code/tile/layout for particular shapes/dtypes. A kernel may do either or both.
5. **Why can dynamic shapes hurt compiler fusion?** Unknown dimensions/layouts make it harder to select fixed tiling, prove legality, or allocate bounded on-chip resources.

8. Comparison Tables

Technique	Kernel fusion	CUDA Graphs
Main action	Combines operations into one kernel	Captures/replays launches
Reduces kernel count	Yes	No
Reduces intermediate HBM traffic	Often	No by itself
Reduces CPU launch overhead	Yes, by fewer launches	Yes, by replay batching
Main risk	Resource pressure/complexity	Shape/address constraints and capture rules
Unfused	Fused	
Easier modular debugging	Better locality potential	
More intermediate writes	Fewer intermediate writes	
More launch boundaries	Fewer launches	
Independent per-op tuning	Joint resource/tile trade-off	

9. Common Mistakes

- Assuming fusion reduces mathematical FLOPs.
- Assuming a framework expression becomes one kernel automatically.
- Confusing CUDA Graph replay with kernel fusion.
- Ignoring register spills and occupancy loss.
- Fusing across a required global synchronization without preserving semantics.
- Benchmarking only kernel time while ignoring launch/transfer changes.

10. Edge Cases / Special Cases

- In-place fusion must respect aliasing and residual values that remain live.
- Numerically sensitive reductions can change rounding due to reordering.
- Dropout fusion during training needs deterministic/random-state handling.
- A fused kernel may have shape/dtype limits and fall back to an unfused path.
- Very large fused kernels can increase compilation time and instruction-cache pressure.

- Multi-GPU collectives create boundaries unless communication-aware kernels overlap/fuse portions safely.

11. How to Explain in Interview

Kernel fusion runs several logical operations in one GPU kernel so intermediates can stay in registers/shared memory and launch overhead is reduced. Transformer examples include GEMM epilogues and fused attention. It helps most when intermediate memory traffic or tiny-kernel launches dominate, but excessive fusion can increase register pressure and reduce occupancy.

12. Quick Revision Notes

- Same model math; fewer kernel/materialization boundaries.
- Primary savings: HBM traffic and launch overhead.
- Epilogue fusion: bias/activation/residual with GEMM store.
- FlashAttention: tiled fusion plus online Softmax.
- CUDA Graphs reduce launch overhead but do not fuse kernels.
- Verify with profiler and end-to-end timing.
- Trap: fusion can lose through spills/occupancy.

13. Practice Tasks

1. Count approximate reads/writes for three element-wise kernels versus one fused kernel.
2. Implement fused and unfused vector bias+ReLU CUDA kernels and benchmark them.
3. Use Nsight Systems to count launches for one Transformer layer.
4. Identify safe GEMM-epilogue candidates in the execution diagram.
5. Explain why a cross-block reduction may force an extra phase/kernel.

14. Final Cheat Sheet

Item	Remember
Core definition	Multiple logical operations executed inside one kernel.
Why it matters	Saves intermediate HBM traffic and launch overhead.
Most asked	Benefits, epilogues, FlashAttention, limits, CUDA Graph distinction.
Main comparison	Fusion changes kernel boundaries; graphs optimize launch replay.
One-line answer	"Fuse producer-consumer operations when it keeps intermediates on-chip without creating harmful resource pressure."

KV Cache

1. Overview

Definition. During autoregressive inference, the KV cache stores the key and value vectors already computed for prior tokens at every Transformer layer. When generating the next token, the model projects only the new token's K/V and attends to the cached history.

Without a KV cache, step t would re-run K/V projections for tokens $0 \dots t-1$ even though those results are unchanged in a causal decoder. The cache trades memory capacity and bandwidth for less repeated computation. It is fundamental to LLM serving, and interviewers ask about it because it connects model semantics, algorithmic complexity, memory sizing, paging, batching, and GQA/MQA.

2. Core Idea

Imagine taking notes during a long interview. When a new question arrives, you consult your notes instead of re-listening to the entire conversation. The notes consume space and must be scanned, but they avoid recomputing history.

For each layer, a common logical layout is:

```
K_cache: [B, H_kv, T_max, d_h]
V_cache: [B, H_kv, T_max, d_h]
```

At decode step t :

1. Compute Q, K, V for the new token only.
2. Apply positional transformation to new Q/K as required.
3. Write new K/V into cache position t .
4. Read cached K positions $0 \dots t$ to compute logits.
5. Read cached V positions $0 \dots t$ to form the weighted sum.
6. Generate the next token and repeat.

Approximate cache bytes for dense storage:

```
KV bytes = 2 * L * B * T * H_kv * d_h * bytes_per_element
           ^ K and V
```

Example: $L=32, B=1, T=4096, H_{kv}=8, d_h=128, \text{BF16}(2 \text{ bytes})$ gives:

```
2 * 32 * 1 * 4096 * 8 * 128 * 2 = 536,870,912 bytes ≈ 512 MiB
```

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Per-layer state	Every layer needs its own historical K and V.	32 layers → 32 cache pairs.	Do not size one layer only.

Subtopic	What it means and why it matters	Example	Common interview angle
Capacity scaling	Linear in batch, tokens, KV heads, head dimension, layers, and dtype bytes.	Formula above.	Exact memory calculation.
MHA/GQA/MQA	Fewer KV heads directly shrink cache size/traffic.	32 Q heads, 8 KV heads → 4× smaller than MHA.	Distinct query vs KV head counts.
Paged allocation	Logical sequences map to fixed-size physical cache blocks.	OS-like page/block table.	Reduces fragmentation/over-reservation.
Prefix caching	Requests sharing an identical prefix may reuse corresponding KV blocks.	common system prompt.	Correctness requires matching model/config/token prefix.
Cache quantization	Store K/V in a lower-precision format with scales.	FP8/INT8 KV.	Capacity/bandwidth vs accuracy/conversion.
Eviction/offload	Move or discard blocks when GPU memory is scarce.	GPU→CPU cache tier.	Transfer latency and recomputation trade-offs.
Beam search sharing	Beams with a common prefix can share immutable blocks, copy-on-write on divergence.	multiple continuations.	Memory sharing semantics.
Cache layout	Token-major, head-major, vectorized, interleaved, or paged layouts target kernel access patterns.	blocks of tokens.	Logical shape is not physical layout.

4. Real-World Example

An LLM server accepts requests whose output lengths are unknown. Reserving a contiguous `T_max` KV buffer for every request wastes memory and fragments allocation. A paged system allocates fixed-size KV blocks only as tokens arrive and keeps a per-sequence block table. Continuous batching can then admit new work whenever enough blocks are free. Attention kernels translate logical token positions through the block table.

5. Diagrams / Mental Models

```
request logical tokens:  0  1  2  3 | 4  5  6  7 | 8  9
                        | block 0 | block 1 | block 2
block table:           [17,      4,      29]

GPU KV pool physical blocks:
... [block 4] ... [block 17] ... [block 29] ...
```

What cache removes	What cache does not remove
Reprojection of old K/V	Reading old K/V during attention
Reprocessing old tokens through earlier blocks	Per-new-token model execution
Duplicate prefix work when prefix reuse is supported	Memory growth with active context

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is KV cache?	Per-layer stored K/V vectors for previously processed tokens.	Autoregressive inference state.	Calling it model weights.
2. Why not cache Q?	Future tokens compare their new Q against old K; old Q is not needed for ordinary causal decode.	Direction of attention reuse.	"Q is too small."
3. What does it save?	Recomputing old tokens' layer states and K/V projections at every step.	Compute-time trade.	Claiming attention becomes constant-time.
4. How does size scale?	$2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_h \cdot \text{bytes}.$	All factors and K/V factor 2.	Using query-head count for GQA.
5. Why can cache become a bottleneck?	It consumes HBM capacity and must be read during decode; both grow with context and concurrency.	Capacity and bandwidth.	Only mentioning storage.
6. How does GQA help?	Fewer KV heads shrink stored K/V and reads while retaining all query heads.	H/H_{kv} ratio.	Reducing Q cache that does not exist.
7. What is PagedAttention?	An attention/memory-management approach that stores KV in fixed-size noncontiguous blocks addressed through tables.	Paging analogy and reduced waste.	Saying it pages model weights.
8. What is prefix caching?	Reusing KV blocks for an identical already-computed token prefix.	Shared immutable prefix.	Reusing semantically similar text.
9. When is cache written?	Prefill writes K/V for prompt tokens; every decode step appends the new token's K/V at every layer.	Lifecycle.	Writing only after the final layer.

Question	Clear answer	Interviewer expects	Common mistake
10. Training vs inference?	Autoregressive serving uses persistent KV across steps; training usually computes full sequences and stores activations for backward instead.	Different state requirements.	Assuming KV cache speeds standard full-sequence training.

7. Deep-Dive Questions

- Why does total generation work remain substantial with a cache?** Generating N tokens sequentially attends over growing contexts $T, T+1, \dots$; cache prevents repeated projections but each step still scans relevant K/V.
- How does paged storage affect kernels?** Address calculation becomes indirect and tokens may cross physical blocks, but memory allocation/utilization and sharing improve.
- What makes prefix cache reuse valid?** Exact token prefix, model weights/version, positional scheme, adapter state, and relevant execution settings must agree so cached K/V are identical.
- Why is KV-cache offload difficult?** Decode needs K/V at low latency every step. PCIe/system-memory bandwidth can be far below local HBM, so transfer/placement must be carefully scheduled.
- How can context parallelism interact with KV cache?** Tokens/cache can be partitioned across devices; each computes partial attention statistics/output and communicates to combine exact results.

8. Comparison Tables

Property	No KV cache	KV cache
Old-token projection	Recomputed each step	Reused
Persistent inference memory	Low	Grows linearly with context/concurrency
Per-step old K/V access	Recomputed then accessed	Read from cache
Decode latency	Much worse as prefix is rerun	Standard practical approach
Contiguous allocation	Paged allocation	
Simple addressing	Block-table indirection	
Can over-reserve max length	Allocates blocks as needed	
External/internal fragmentation risk	Small bounded block-level waste	
Harder prefix/beam sharing	Natural block sharing/copy-on-write	

9. Common Mistakes

- Forgetting cache exists at every layer.
- Caching queries unnecessarily.
- Saying cache makes decode $O(1)$ in context length.
- Using H instead of H_{kv} for GQA/MQA sizing.
- Forgetting the factor of two for K and V or dtype bytes.

- Assuming paged KV means moving data to disk.
- Treating similar prompts as safely cache-compatible.

10. Edge Cases / Special Cases

- Sliding-window models may evict tokens outside the attention window, though some layers can use different rules.
- Speculative decoding may allocate cache for draft tokens and roll back/reclaim rejected suffixes.
- Beam search shares prefixes but diverging beams need copy-on-write behavior.
- RoPE scaling or position changes affect whether cached K remains valid.
- Cancellation must promptly reclaim blocks without racing active kernels.
- Mixed batches need per-sequence lengths so attention ignores unused cache slots.
- Cross-attention K/V from an encoder can be cached separately and remain fixed across decoder steps.

11. How to Explain in Interview

The KV cache stores each layer's past key and value vectors, so autoregressive decode projects only the new token. The new query still scans cached keys and values, so per-step attention grows with context and becomes memory-bandwidth-heavy. Cache size is $2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_h \cdot \text{bytes}$, and paging/GQA reduce allocation waste or the amount stored.

12. Quick Revision Notes

- Store K and V, not Q, for every layer and token.
- Size: $2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_h \cdot \text{bytes}$.
- Prefill populates cache; decode appends one position per step.
- Saves old-token recomputation, not old-cache reading.
- GQA/MQA reduce distinct KV heads.
- Paging improves allocation/sharing; it does not mean disk by default.
- Trap: long-context decode remains $O(T)$ attention work per new token.

13. Practice Tasks

1. Compute KV bytes for an architecture in MHA, GQA, and MQA forms.
2. Draw a paged cache block table for three unequal sequences.
3. Simulate append, cancellation, and block reclamation.
4. Explain why Q is not reusable for a future query.
5. Estimate how many concurrent requests fit after model weights reserve part of HBM.

14. Final Cheat Sheet

Item	Remember
Core definition	Per-layer historical K/V stored across generation steps.
Why it matters	Makes autoregressive inference practical but consumes capacity/bandwidth.
Most asked	Why K/V, size formula, GQA, paging, prefix reuse.

Item	Remember
Main comparison	Cache removes recomputation, not the context scan.
One-line answer	"Cache every layer's past K/V, append the new token, and trade extra HBM for avoiding repeated prefix computation."

Memory Bandwidth

1. Overview

Definition. Memory bandwidth is the rate at which data can be moved between a memory level and GPU compute units, commonly measured in bytes per second. For Transformer execution, the most discussed limit is HBM/global-memory bandwidth, but L2, shared-memory, register, host-device, and inter-GPU bandwidth also matter.

A kernel is bandwidth-bound when time is primarily limited by moving required bytes rather than by arithmetic throughput. LLM decode often streams large weight matrices and KV caches for few active tokens, while large prefill GEMMs reuse weights enough to become compute-bound. Interviewers ask this topic to test roofline reasoning, arithmetic intensity, coalescing, reuse, and why nominal FLOPs do not predict latency.

2. Core Idea

A GPU has a very fast factory (Tensor Cores) and a road delivering raw material (memory). If the road cannot deliver operands quickly enough, faster machinery sits idle.

Two lower bounds help:

```
compute_time ≥ required_FLOPs / peak_or_sustained_FLOP_rate
memory_time  ≥ required_bytes / sustained_bandwidth
kernel_time  ≥ max(compute_time, memory_time)    # simplified roofline view
```

Arithmetic intensity is:

```
AI = FLOPs / bytes transferred from the memory level being analyzed
```

Small decode example: multiplying one row by a large weight matrix uses each weight roughly once, about two arithmetic operations per weight for multiply-add but at least one weight load. BF16 weights therefore provide roughly $2 \text{ FLOPs} / 2 \text{ bytes} = 1 \text{ FLOP/byte}$ before counting other traffic. Large prefill batches reuse the same weight across many token rows, increasing intensity.

Step-by-step optimization reasoning:

1. Measure kernel time and achieved bandwidth/compute throughput.

2. Count unavoidable and actual bytes at the relevant memory level.
3. Identify reuse that caches/shared memory/register tiling can capture.
4. Coalesce accesses so memory transactions carry useful data.
5. Reduce bytes using fusion, smaller dtype, GQA, or avoiding materialization.
6. Re-measure; a bottleneck can move after optimization.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Arithmetic intensity	FLOPs per byte moved at a chosen level.	batch-one GEMV has low AI.	Roofline classification.
Coalescing	Warp lanes access addresses combined into efficient memory transactions.	adjacent lanes load adjacent features.	Coalescing improves transaction efficiency, not reuse.
Reuse	Load data once, use it multiple times before eviction.	GEMM weight tile used by many rows.	Tiling raises AI.
HBM/global memory	Large off-chip device memory storing weights, activations, cache.	model weights.	High capacity/bandwidth but slower than on-chip storage.
L2 cache	Shared on-device cache across SMs.	reused weight/cache lines.	Hit rate reduces HBM traffic.
Shared memory/registers	Explicit/implicit on-chip storage with high bandwidth and limited capacity.	attention/GEMM tiles.	Resource pressure/occupancy.
Quantization	Fewer bytes per weight/cache/activation.	BF16→INT8 halves stored bytes.	Only helps if supported and accuracy/overhead acceptable.
Effective bandwidth	Useful bytes divided by elapsed time; distinguish from advertised peak.	bytes/time benchmark.	Include read/write traffic and synchronization.
Interconnect bandwidth	NVLink/PCIe/network limits tensor-parallel collectives or offload.	all-reduce.	Multi-GPU can be communication-bound.

4. Real-World Example

Suppose one decode layer must read roughly 200 MiB of weights and 16 MiB of KV data for an active token group, while element-wise activations are small. Even a GPU with 2 TB/s ideal HBM bandwidth needs about $216 \text{ MiB} / 2 \text{ TB/s} \approx 0.11 \text{ ms}$ as a theoretical transfer floor for that layer. Real latency is higher due to imperfect bandwidth utilization, compute, cache misses, launches, and dependencies. Across many layers, streaming weights dominates unless batch reuse, quantization, or caching changes the traffic.

5. Diagrams / Mental Models

capacity ↑, latency usually ↑, bandwidth per byte of storage usually ↓

Registers	tiny, thread-local, fastest
Shared memory/L1	small, on each SM
L2	larger, shared by SMs
HBM/global	GBs, device-wide
Host memory	larger, across PCIe/NVLink/CXL path
Storage	much slower for active inference state

Roofline mental model:

$$\text{attainable FLOP/s} = \min(\text{compute ceiling}, \text{arithmetic intensity} \times \text{memory bandwidth})$$

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is memory bandwidth?	Bytes that can be transferred per unit time between a memory level and compute.	Specify level and sustained vs peak.	Calling it memory capacity.
2. What is bandwidth-bound?	Additional arithmetic units would not help because operand/result movement limits execution.	Bottleneck definition.	"Uses lots of memory."
3. What is arithmetic intensity?	FLOPs performed per byte transferred at the memory level under study.	Units and level dependence.	FLOPs divided by element count.
4. Why is decode often bandwidth-bound?	Few token rows give little reuse while large weights and growing KV cache are streamed.	Small M , low AI.	Only saying decode is sequential.
5. Why can prefill be compute-bound?	Many token rows reuse weight tiles and create large GEMMs with high AI.	Reuse across $B \cdot S$.	Assuming prefill is always compute-bound.
6. How does fusion help bandwidth?	It eliminates intermediate HBM writes and reads.	Bytes saved, not necessarily FLOPs.	Saying it increases physical bandwidth.
7. How does quantization help?	Smaller representations reduce bytes and may unlock specialized compute, with scale/accuracy overheads.	Conditional benefit.	Assuming exact bit-ratio speedup.

Question	Clear answer	Interviewer expects	Common mistake
8. What is coalescing?	Combining warp lanes' nearby accesses into efficient memory transactions.	Access pattern.	Confusing it with cache reuse.
9. How do you estimate a lower bound?	bytes / sustained bandwidth, then compare with FLOPs / sustained compute.	Roofline reasoning.	Using peak specs as guaranteed time.
10. Can high occupancy fix bandwidth limits?	It can help hide latency until bandwidth saturates; it cannot exceed the bandwidth ceiling.	Latency hiding vs throughput.	"More warps always make it faster."

7. Deep-Dive Questions

- At which memory level is arithmetic intensity defined?** Any chosen level. A tiled GEMM can have high HBM intensity but different shared-memory/register traffic; always state the boundary.
- Why can measured bandwidth exceed a naive useful-byte estimate?** Hardware transactions, cache behavior, write allocation, repeated loads, and measurement definitions may differ; distinguish requested/useful from actual traffic.
- How does batching increase projection AI?** A weight tile loaded once is multiplied by more activation rows, increasing FLOPs faster than weight bytes.
- Why can lower precision become compute-bound after being bandwidth-bound?** Reducing operand bytes raises effective AI; specialized low-precision units also change the compute ceiling, so the limiting roofline can shift either way.
- Why is random/noncontiguous KV access costly?** It reduces transaction efficiency and cache locality, though paging benefits allocation; kernels use block layouts and vectorization to limit the penalty.

8. Comparison Tables

Compute-bound kernel	Bandwidth-bound kernel
Limited mainly by arithmetic throughput	Limited mainly by bytes moved
Higher FLOP utilization	Compute units may wait for data
Improve math pipeline/tiling/instruction mix	Reduce bytes, improve locality/coalescing/reuse
Large GEMM often trends here	Small-batch GEMV/decode often trends here
Metric	Meaning
Capacity (GB)	How much data fits
Bandwidth (GB/s or TB/s)	How fast data can move
Latency (ns/cycles)	Delay before a particular access completes
Arithmetic intensity (FLOP/byte)	Work obtained per byte moved
Utilization (%)	Fraction of a specified hardware ceiling achieved

9. Common Mistakes

- Confusing memory bandwidth, capacity, and latency.
- Using advertised peak bandwidth as sustained application bandwidth.
- Ignoring writes or intermediate tensor traffic in byte counts.
- Saying coalescing creates reuse.
- Assuming high occupancy implies high performance.
- Labeling a whole model bandwidth-bound when phases/kernels differ.
- Forgetting inter-GPU and host-device links have separate ceilings.

10. Edge Cases / Special Cases

- Tiny kernels may be launch-bound before either roofline matters.
- Cache-resident weights can make HBM byte estimates pessimistic for small models/repeated layers, but do not assume residency without measurement.
- Unified-memory page faults can dominate and invalidate a simple HBM model.
- ECC, clocks, power limits, thermals, concurrent work, and access efficiency affect sustained bandwidth.
- Sparse/quantized formats may save bytes but add metadata and irregular decoding.
- Multi-tenant workloads compete for cache and bandwidth.

11. How to Explain in Interview

Memory bandwidth is the byte rate between a memory level and compute. A kernel is bandwidth-bound when $\text{bytes}/\text{bandwidth}$ exceeds its compute-time floor. Decode often has low arithmetic intensity because it streams large weights and KV cache for few token rows; batching, fusion, quantization, coalescing, and tiling improve reuse or reduce bytes.

12. Quick Revision Notes

- Bandwidth is rate; capacity is amount; latency is delay.
- $\text{AI} = \text{FLOPs}/\text{bytes}$ at a stated memory boundary.
- Roofline: $\min(\text{compute ceiling}, \text{AI} \times \text{bandwidth})$.
- Prefill GEMMs reuse weights; small-batch decode often cannot.
- Fusion removes intermediate traffic; quantization shrinks operands.
- Coalescing improves transactions; tiling creates reuse.
- Trap: occupancy hides latency but cannot raise bandwidth ceiling.

13. Practice Tasks

1. Estimate $\text{bytes}/\text{bandwidth}$ and $\text{FLOPs}/\text{throughput}$ for a projection.
2. Calculate AI for matrix-vector versus matrix-matrix multiplication under a simple traffic model.
3. Profile achieved HBM bandwidth with Nsight Compute.
4. Compare strided and coalesced CUDA loads.
5. Recalculate KV-cache traffic after changing MHA to GQA.

14. Final Cheat Sheet

Item	Remember
Core definition	Rate at which operands/results move through a memory boundary.
Why it matters	Often sets the latency/throughput ceiling for LLM decode.
Most asked	Roofline, AI, decode vs prefill, coalescing, quantization.
Main comparison	Compute-bound needs more math throughput; bandwidth-bound needs fewer/more useful bytes.
One-line answer	"Compare FLOPs/compute rate with bytes/bandwidth; the larger floor identifies the likely bottleneck."

Prefill vs Decode

1. Overview

Definition. Autoregressive LLM inference has two distinct execution phases:

- **Prefill:** process the prompt tokens, produce their hidden states, and populate the KV cache.
- **Decode:** repeatedly process newly selected tokens, append their K/V, and generate one next-token decision per active sequence per iteration.

The layers and weights are the same, but tensor shapes and bottlenecks are very different. Prefill exposes many token rows and dense attention; decode has a small query dimension, persistent cache reads, and a serial dependency between generated tokens. Production serving schedulers, kernels, batching policies, and metrics are designed around this split. Interviewers ask it because it is the core systems insight behind LLM inference performance.

2. Core Idea

Suppose the prompt is "The capital of France is" and the model generates "Paris".

```
Prefill:
process [The, capital, of, France, is] together
write K/V for all five tokens at every layer
produce logits for the next token
```

```
Decode step 1:
select/process [Paris]
read prior cache, append Paris K/V
produce logits for the following token
```

```
Decode step 2:
process only the token selected at step 1
repeat
```

Prefill can parallelize computations across prompt positions because every input token is already known; a causal mask preserves semantics. Decode cannot compute token $t+1$ until sampling has selected token t , producing an unavoidable loop at the model level.

Performance metrics:

- **TTFT (time to first token):** queueing plus prefill plus first sampling/output overhead.
- **TPOT (time per output token):** spacing between generated tokens, often reported after the first.
- **Inter-token latency:** similar user-visible decode cadence measure.
- **Throughput:** prompt tokens/s, output tokens/s, or total tokens/s—state which.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Prefill GEMMs	$M=B \cdot S$ token rows make projections/MLPs matrix-matrix operations.	$[4096, 4096] @ [4096, N]$.	High reuse and compute utilization.
Prefill attention	Many queries attend within the prompt; dense causal work grows quadratically in prompt length.	FlashAttention tiles $S \times S$.	Compute/IO and TTFT.
Decode GEMMs	Usually $M=B$; one new row per active sequence.	$[B, D] @ [D, N]$.	Low weight reuse at small batch.
Decode attention	One query per request attends to T cached positions.	$[1, d_h] @ [d_h, T]$.	KV bandwidth grows with context.
Serial generation	Next input token depends on current logits and sampling.	cannot generate token 12 before token 11.	Limits single-request parallelism.
Chunked prefill	Divide long prompts into token chunks and interleave them with decode work.	process 512 prompt tokens per scheduling round.	TTFT/TPOT fairness and utilization.
Disaggregated serving	Different workers specialize in prefill and decode; KV state must transfer.	prefill GPU $\rightarrow$ decode GPU.	Specialization vs KV transfer cost.
Scheduling	Mix prompt chunks and decode tokens subject to token/cache budgets.	continuous batching.	Head-of-line blocking and SLOs.
Sampling	Logits processing/top-k/top-p produces the next token between decode iterations.	GPU or CPU sampling.	Sequential dependency and synchronization.

4. Real-World Example

A chat service receives one 16k-token document prompt while ten users are already decoding. If it runs the whole prefill as one batch, the large operation may delay decode iterations and cause visible pauses. A chunked-prefill scheduler splits the document into smaller chunks and interleaves them with decode tokens. This may increase the long prompt's TTFT slightly but protects existing users' TPOT and keeps GPU work batched.

5. Diagrams / Mental Models



Characteristic	Prefill	Decode
Known input tokens	Entire prompt	Only current selected token
Query length per request	Many	Usually 1
KV action	Bulk populate	Append one position
Projection/MLP shape	Large GEMM	Small- M GEMM/GEMV-like
Typical pressure	Compute and attention IO	Weight/KV bandwidth, launches, serial latency
User metric	TTFT	TPOT/inter-token latency

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is prefill?	Full/chunked prompt processing that builds per-layer KV cache and first-token logits.	Prompt phase and cache creation.	Calling tokenization prefill.
2. What is decode?	Iterative new-token execution using cached K/V, normally one new token per sequence each step.	Autoregressive loop.	Rerunning the whole prompt.
3. Why is prefill more parallel?	All prompt token inputs are known, so positions can be evaluated together under a causal mask.	Parallel compute with causal semantics.	Saying tokens attend to the future.
4. Why is decode sequential?	Token t+1 cannot be chosen until logits for token t are produced and sampled.	Data dependency across iterations.	Saying GPU kernels themselves are serial.

Question	Clear answer	Interviewer expects	Common mistake
5. Which is compute-bound?	Large prefill often trends compute-bound; small-batch decode often trends bandwidth-bound, but measure for actual shapes/hardware.	Qualified roofline reasoning.	Absolute claim for every workload.
6. What does KV cache change?	Prefill writes history once; decode reuses it and appends new K/V, avoiding prefix recomputation.	Cache lifecycle.	Claiming no history is read.
7. What is TTFT?	Arrival-to-first-token latency, including queueing/prefill and service overhead.	User-facing latency.	Measuring only GPU prefill time.
8. What is TPOT?	Average/percentile time between output tokens after the first, with definition stated.	Decode cadence.	Reporting total latency divided by all tokens without clarification.
9. Why chunk prefill?	To bound long-prompt scheduling blocks and interleave decode, balancing TTFT, TPOT, and utilization.	Service-level trade-off.	Saying it changes model outputs inherently.
10. What is prefill/decode disaggregation?	Run phases on specialized workers and transfer KV cache between them.	Resource specialization plus transfer cost.	Forgetting cache movement.

7. Deep-Dive Questions

- Why can total prefill attention be quadratic while cache storage is linear?** Attention compares token pairs (S^2), but it stores one K and one V vector per token (S).
- How does continuous batching alter decode GEMM shape?** It groups one token from each currently active request, making M the number of scheduled tokens rather than one.
- When can disaggregation lose?** If KV transfer, network contention, queueing, or loss of locality exceeds benefits from phase specialization.
- Why can very long prefill hurt other requests?** Large kernels occupy the GPU for longer non-preemptive intervals and consume token/cache budgets, creating head-of-line blocking.
- How does speculative decoding relax serial cost?** A cheaper draft proposes several tokens and the target verifies them together, creating larger target-model work per iteration, but accepted tokens still determine correctness and benefit.

8. Comparison Tables

Metric	Optimized primarily by	Possible tension
TTFT	Short queue, fast/large prefill processing	Large prefill batches can delay admission

Metric	Optimized primarily by	Possible tension
TPOT	Frequent decode scheduling, efficient cache reads	Too much prefill work causes decode pauses
Throughput	Larger batches, high utilization	Queueing/batching can raise latency
Capacity	Efficient KV allocation/precision	More concurrent requests increase bandwidth contention
Monolithic prefill		Chunked prefill
Fewer scheduling boundaries		Easier interleaving/fairness
Potentially best isolated prompt throughput		Better TPOT protection under mixed load
Can block decode for long periods		Extra scheduling/chunk-boundary overhead

9. Common Mistakes

- Saying prefill predicts every prompt token sequentially.
- Saying decode is single-threaded; the model step is highly parallel internally, while steps are sequentially dependent.
- Calling KV cache a constant-time attention optimization.
- Mixing prompt tokens/s and output tokens/s.
- Ignoring queueing in TTFT.
- Assuming the same batch size/shape policy is optimal for both phases.
- Recommending disaggregation without accounting for KV transfer.

10. Edge Cases / Special Cases

- Prefix-cache hits can skip some or nearly all prefill computation but still require valid cache placement.
- Chunk boundaries must preserve positions and causal visibility exactly.
- Empty prompts still require special beginning-of-sequence/input handling.
- Beam/speculative decoding may process multiple candidate tokens per sequence per iteration.
- Encoder-decoder models can precompute encoder state and cross-attention K/V separately.
- Very short prompts may not create compute-bound prefill; launch overhead can still dominate.
- Sampling on CPU can add synchronization and host-device transfer latency if poorly integrated.

11. How to Explain in Interview

Prefill processes all known prompt tokens in parallel, creates the KV cache, and mainly determines time to first token. Decode processes one newly selected token per active sequence, reads and appends KV state, and repeats serially, determining inter-token latency. Prefill uses large efficient GEMMs; decode has small-M GEMMs and growing KV reads, so it is often bandwidth-sensitive.

12. Quick Revision Notes

- Prefill: prompt → bulk compute → populate cache → first logits.
- Decode: one scheduled new token/request → read cache → append cache → sample → repeat.
- Prefill exposes token parallelism; generation steps are dependent.

- TTFT maps mainly to queue + prefill; TPOT maps mainly to decode cadence.
- Chunking trades isolated prefill speed for fairness/TPOT control.
- Disaggregation adds KV transfer.
- Trap: “decode is serial” refers to token-step dependency, not absence of GPU parallelism.

13. Practice Tasks

1. Draw shapes for a 1024-token prefill and the next three decode iterations.
2. Build a timeline for two requests arriving at different times under static and continuous batching.
3. Separate TTFT into queue, tokenize, H2D, prefill, sampling, and streaming components.
4. Explain when chunk size should be reduced.
5. Calculate cumulative attention pair counts for N generated tokens after prompt length P .

14. Final Cheat Sheet

Item	Remember
Core definition	Prefill builds prompt state; decode iteratively extends it.
Why it matters	Same model, radically different shapes, bottlenecks, and metrics.
Most asked	Parallelism, KV lifecycle, TTFT/TPOT, chunking, bottlenecks.
Main comparison	Prefill: large token-parallel GEMMs; decode: serial steps with small GEMMs/cache scans.
One-line answer	“Prefill computes the prompt and fills KV; decode reuses KV to generate one dependent step at a time.”

Batch Size

1. Overview

Definition. Batch size is the number of independent examples or sequences processed together, but LLM serving needs a more precise definition. A prefill batch may contain many prompt tokens per sequence, while a decode iteration normally schedules one token from each active sequence. Engines therefore also use **token budget**, the total scheduled tokens in an iteration.

Batching matters because it amortizes weight loads, kernel launches, and fixed overhead, increasing throughput and arithmetic intensity. It also consumes more activation/KV memory and can increase queueing and per-request latency. Interviewers ask about it to see whether candidates distinguish throughput from latency, static from continuous batching, and sequence count from token count.

2. Core Idea

Think of a delivery truck. Sending one package per trip gives low latency for the first package but poor fuel efficiency. Waiting to fill the truck improves packages per trip but makes early packages wait.

For a projection $Y=XW$:

```
batch/token rows M=1:  [1,D]@[D,N]  each weight mostly used once
batch/token rows M=32: [32,D]@[D,N] one loaded weight tile serves 32 rows
```

The weight matrix is unchanged, so increasing M raises computation per weight byte. However, in an online service the requests must first exist and may have different lengths.

Step by step in continuous batching:

1. Scheduler tracks waiting prefills and active decodes.
2. It selects work subject to maximum sequences, maximum scheduled tokens, and free KV blocks.
3. Finished requests leave immediately.
4. New requests join later iterations instead of waiting for an entire static batch to finish.
5. Shapes change each iteration, so kernels use metadata for lengths/cache blocks.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Sequence batch size	Number of requests/sequences grouped.	32 active decodes.	Not equal to token count in prefill.
Token batch size	Total tokens scheduled in a forward pass/chunk.	8 prompts $\times$ 512 tokens = 4096 tokens.	Better compute-shape descriptor.
Static batching	Fixed group runs until all finish.	offline evaluation batch.	Tail requests cause padding/idle slots.
Continuous/in-flight batching	Add/remove requests between iterations.	production generation server.	Better utilization for variable output lengths.
Dynamic batching window	Wait briefly to collect requests.	2 ms queue window.	Throughput-latency trade-off.
Padding/packing	Equalize shapes with padding or store useful tokens compactly.	ragged prompts.	Masking does not necessarily skip padded work.
Memory limit	More requests mean more activations and persistent KV allocation.	cache blocks cap concurrency.	OOM/capacity planning.
SLO-aware scheduling	Batch choices respect TTFT and TPOT targets.	prioritize overdue decode.	Maximum throughput is not always desired.
Bucketing	Group similar lengths to reduce padding/imbalance.	prompts 1–512 together.	Useful for static/offline workloads.

4. Real-World Example

An offline summarization job can use a large static batch and maximize tokens/s because individual response latency is unimportant. An interactive chatbot instead uses continuous batching: each decode iteration groups one token from many users, while new prompt chunks are admitted within a token budget. The server may cap the batch even before HBM is full to preserve TPOT targets.

5. Diagrams / Mental Models

Static batch:

A: [d][d][done][idle][idle]

B: [d][d][d][d][done] next batch waits for B

Continuous batch:

A: [d][d][done]

B: [d][d][d][d][done]

C: [p][d][d][d] C enters freed slot

Increasing batch can...	But can also...
Reuse weights across more rows	Increase queueing latency
Fill more SMs/Tensor Cores	Increase KV/activation memory
Amortize launches	Mix uneven sequence lengths
Raise throughput	Worsen tail latency/TPOT at saturation

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why does batching improve GPU throughput?	It creates larger operations, reuses weights across token rows, fills hardware, and amortizes launches.	AI/utilization.	"Because more GPUs are used."
2. Does larger batch always reduce latency?	No. Service queueing and more work per iteration can increase per-request latency even if throughput improves.	Throughput-latency trade-off.	Equating throughput with latency.
3. What is continuous batching?	Requests join and leave between generation iterations rather than waiting for a fixed batch to complete.	Variable-length serving.	Changing membership inside a running kernel.
4. Sequence count vs token count?	Sequence count is requests; token count includes prompt chunks or one decode token per active sequence.	Precise batch definition.	Calling both B .
5. Why is static batching inefficient for generation?	Output lengths vary; completed sequences leave padding/idle slots until the longest finishes.	Tail effect.	Only mentioning input padding.

Question	Clear answer	Interviewer expects	Common mistake
6. What limits maximum batch?	KV/activation memory, token budget, kernel limits, latency SLOs, and scheduler policy.	More than OOM.	Hardware thread limit alone.
7. How does batch affect decode projections?	It changes GEMM M from very small toward a larger value, improving weight reuse.	Bandwidth-to-compute shift.	Saying model weights shrink.
8. Why bucket lengths?	Similar lengths reduce padding and load imbalance.	Practical batching.	Treating it as required for paged kernels.
9. How does batching affect KV cache?	More concurrent sequences multiply cache capacity and bandwidth demand; lengths differ per sequence.	$B \cdot T$ scaling.	One shared cache for batch.
10. How would you tune batch size?	Sweep under representative arrival/length distributions and report throughput plus TTFT/TPOT percentiles and memory.	Measure workload/SLO.	Tune only a synthetic fixed length.

7. Deep-Dive Questions

- Why can throughput eventually fall as batch increases?** Capacity pressure, cache thrashing, padding, longer scheduling rounds, less favorable tactics, or communication contention can exceed reuse benefits.
- What is iteration-level batching?** Every decode iteration schedules the current active token set; membership and context lengths may differ from the prior iteration.
- How do mixed prefill/decode batches complicate kernels?** Tokens have different query lengths and cache behavior, so engines may use specialized kernels, chunking, or separate sub-batches while coordinating one schedule.
- Why does a token budget help more than only max batch size?** One long prefill can contain far more work than many one-token decodes; tokens better bound compute and temporary memory.
- How does tensor parallelism change optimal batch?** Communication is paid per layer; larger local work can amortize collectives, while sharding also reduces per-rank GEMM sizes and may require more batch to stay efficient.

8. Comparison Tables

Static batching	Continuous batching
Fixed membership for full generation	Membership changes at iteration boundaries
Simple	Scheduler/cache metadata complexity
Padding/idle waste from unequal outputs	Freed capacity reused quickly
Suitable for uniform offline work	Preferred for online variable-length serving

Latency-oriented policy	Throughput-oriented policy
Short waiting window/smaller batches	Wait/accumulate more work
Frequent decode scheduling	Larger token batches
Lower queue delay	Better weight/launch amortization
Lower peak utilization possible	Higher tail latency possible

9. Common Mistakes

- Giving “batch size” without saying sequences or tokens.
- Assuming maximum batch yields maximum useful throughput.
- Ignoring queueing and tail latency.
- Treating continuous batching as mathematically changing attention.
- Believing padding masks make padded compute free.
- Forgetting cache memory scales with concurrent sequence lengths.
- Benchmarking batch sizes with unrealistic identical prompts/outputs only.

10. Edge Cases / Special Cases

- Batch 0 should not launch normal compute kernels.
- Batch 1 can still use large prefill token count and efficient GEMMs.
- One unusually long sequence can dominate padding or attention work.
- Request cancellation/failure changes membership and must safely reclaim cache.
- Beam search creates multiple logical sequences that may share prefix blocks.
- Multi-tenant priorities/fairness may deliberately sacrifice peak throughput.
- CUDA Graph replay may require a finite set of padded/captured batch shapes.

11. How to Explain in Interview

Batching groups independent token work so large GEMMs reuse weights and amortize launches. In LLM serving I distinguish sequence batch size from scheduled-token count. Continuous batching replaces finished requests between decode iterations, improving utilization for variable lengths, but larger batches increase KV memory, queueing, and iteration latency, so tuning must respect TTFT/TPOT SLOs.

12. Quick Revision Notes

- Sequence batch = requests; token batch = total scheduled tokens.
- Larger M improves projection weight reuse.
- Static batch waits for tail; continuous batch replaces finished work.
- Batching improves throughput but can hurt queue/tail latency.
- KV capacity and token budget limit concurrency.
- Tune on realistic length/arrival distributions.
- Trap: batch 1 prefill may still contain thousands of token rows.

13. Practice Tasks

1. Simulate static and continuous batching for outputs of lengths [2,5,3].
2. Calculate total scheduled tokens for mixed prompt chunks and decodes.
3. Plot throughput, TTFT p99, and TPOT p99 over batch-size sweeps.
4. Estimate KV capacity for 1, 8, and 64 concurrent sequences.
5. Explain when a batching wait window helps and when it violates an SLO.

14. Final Cheat Sheet

Item	Remember
Core definition	Independent sequences/tokens grouped into GPU work.
Why it matters	Raises utilization/reuse but costs memory and latency.
Most asked	Static vs continuous, tokens vs sequences, throughput vs latency.
Main comparison	Static is simple but tail-wasteful; continuous reuses freed slots.
One-line answer	"Batch enough token rows to reuse weights, but cap scheduling by KV capacity and latency targets."

Sequence Length

1. Overview

Definition. Sequence length is the number of token positions processed or retained as context. During prefill, the prompt length controls how many query and key/value positions are computed. During decode, the context length T is the prompt plus accepted/generated tokens visible to the new query.

Sequence length matters because different Transformer components scale differently. Projections and dense MLPs are linear in token count; dense prefill attention compares token pairs and is quadratic; KV-cache capacity is linear; and each decode step's attention scan is linear in current context. It is used in context-window design, serving admission, batching, kernel selection, and memory planning. Interviewers ask it to test careful complexity analysis rather than the slogan "Transformers are quadratic."

2. Core Idea

If a prompt doubles from S to $2S$:

- token-wise projection/MLP work roughly doubles;
- stored KV entries double;
- dense full self-attention pair count grows from S^2 to $(2S)^2=4S^2$;
- a decode query after that prompt scans twice as many cached positions.

For a causal prefill, valid pairs number $S(S+1)/2$, still $\theta(S^2)$. A kernel may skip masked blocks, but the asymptotic order remains quadratic.

Step-by-step shape propagation:

X:	[B,S,D]	$O(S)$ elements
Q/K/V:	[B,H or H_kv,S,d_h]	$O(S)$
scores:	[B,H,S,S] logically	$O(S^2)$
attention out:	[B,S,D]	$O(S)$
KV cache:	per layer K and V	$O(T)$
decode scores:	[B,H,1,T]	$O(T)$ each step

Efficient attention can avoid materializing the S^2 matrix, making memory usage closer to linear for forward activations, but it still evaluates dense token pairs unless the attention pattern itself is sparse/windowed.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Prompt length	Tokens processed in prefill.	$S=8192$.	TTFT and S^2 attention work.
Context length	Tokens visible to a decode query.	prompt + generated history.	KV reads and capacity.
Output length	Number of generated tokens.	max new tokens 512.	Repeated serial steps and cumulative work.
Model context window	Maximum supported positions under model/runtime configuration.	32k tokens.	Architectural limit vs available memory.
Causal triangle	Only earlier/current positions are valid.	$S(S+1)/2$ pairs.	Still quadratic asymptotically.
Sliding window	Attend only to most recent W tokens.	$W=4096$.	Per-step attention becomes $O(W)$ after window fills.
Sparse/block attention	Predetermined or data-dependent subset of pairs.	local + global blocks.	May reduce arithmetic, requires kernel/model support.
Position encoding	Encodes order and may constrain length generalization.	RoPE.	Runtime accepting length does not guarantee quality.
Chunking	Processes prompt pieces to meet scheduler/memory constraints while preserving cache/positions.	512-token chunks.	Does not automatically remove total dense causal work.
Length bucketing	Groups similar lengths.	[1-512], [513-1024].	Reduce padding waste.

4. Real-World Example

A retrieval application increases prompt length from 4k to 32k tokens. It must account for more than an $8\times$ larger input tensor. Dense prefill attention has about $64\times$ as many pair comparisons, KV cache is $8\times$ larger, and every later decode token scans roughly $8\times$ more history. FlashAttention controls intermediate memory traffic, but does not make dense pairwise arithmetic linear. The team may instead retrieve fewer passages, summarize, use sliding-window/sparse architecture, quantize cache, or provision more hardware.

5. Diagrams / Mental Models

$S = 4$: valid causal pairs = 10

```

• x x x
• • x x
• • • x
• • • •

```

$S = 8$: valid causal pairs = 36, not 20

Quantity	Scaling when length doubles
Token-wise activations/projections/MLP	about $2\times$
KV-cache capacity	$2\times$
One decode attention scan	$2\times$
Dense full prefill attention pairs	$4\times$

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why is attention called quadratic?	Dense full self-attention compares every query position with every key position, giving S^2 pairs.	Pair matrix.	Saying all Transformer operations are quadratic.
2. Are projections quadratic in S ?	No. QKV/output/MLP process each token row, so their work is linear in S for fixed widths.	Component-wise complexity.	Applying S^2 to the whole block.
3. How does KV-cache size scale?	Linearly with stored tokens T , times layers, KV heads, head width, batch, dtype, and K/V.	Size formula.	Calling it quadratic.
4. What is decode complexity per new token?	Projection/MLP work is largely independent of T ; dense attention reads/operates over T , so attention is $O(T)$.	Component separation.	Calling entire step $O(T)$ without qualification or $O(1)$ due to cache.

Question	Clear answer	Interviewer expects	Common mistake
5. Total cost of generating N tokens?	Decode attention sums over growing contexts: $\sum_{i=0}^{N-1} (P+i)$, plus N projection/MLP passes.	Arithmetic-series reasoning.	$O(N)$ for everything.
6. Does causal masking make attention linear?	No. It roughly halves valid pairs but remains $O(S^2)$.	Constants vs asymptotics.	Confusing triangular with diagonal.
7. Does FlashAttention make compute linear?	No for dense exact attention; it reduces HBM IO/materialization using tiling and online Softmax.	IO vs FLOP complexity.	Saying it is sparse/approximate.
8. What is a context window?	Maximum number of token positions the model/runtime can use, constrained by architecture/configuration and resources.	Not synonymous with current length.	Assuming advertised window always fits every batch.
9. How does sliding-window attention help?	It restricts each query to at most w recent keys, bounding cache/attention according to architecture semantics.	$O(Sw)$ prefill, $O(w)$ step after warmup.	Applying it to a model not trained/configured for it.
10. Why bucket by length?	It reduces padding and improves regularity/utilization.	Practical batching.	Claiming it changes model complexity.

7. Deep-Dive Questions

- When does MLP work dominate attention despite long sequences?** Wide models have very large dense projections; at moderate S , $D^2/D \cdot D_{\text{ff}}$ terms may exceed attention's $S^2 \cdot D$. Dominance depends on dimensions and hardware, not complexity notation alone.
- Why can long-context decode latency rise even with enough HBM capacity?** Every new query reads and processes more cached K/V, increasing bandwidth and reduction work.
- Can chunked prefill reduce peak memory without reducing total work?** Yes. It bounds scheduled query tokens/intermediates and improves scheduling, but exact dense causal relationships still need to be evaluated across chunks.
- Why can position extrapolation fail before memory does?** The model's learned/trained positional behavior may degrade outside its training regime even if the runtime can allocate larger buffers.
- How does GQA affect sequence-length scaling?** The order stays linear for cache and quadratic/linear for attention phases, but fewer KV heads reduce constants for cache capacity/bandwidth.

8. Comparison Tables

Attention pattern	Visible keys/query	Prefill pair work	Decode scan after warmup
Dense bidirectional	S	S^2	not typical autoregressive decode

Attention pattern	Visible keys/query	Prefill pair work	Decode scan after warmup
Dense causal	up to S	$S(S+1)/2$	T
Sliding window W	up to W	about $S \cdot W$	up to W
Block sparse	pattern-dependent	nonzero blocks only	pattern-dependent
Term	Dependence on sequence/context length		
QKV projection	$O(S \cdot D \cdot N_{\text{qkv}})$		
Prefill QK/PV	$O(H \cdot S^2 \cdot d_h)$ each for dense self-attention		
MLP	$O(S \cdot D \cdot D_{\text{ff}})$ with architecture constants		
KV storage	$O(L \cdot T \cdot H_{\text{kv}} \cdot d_h)$		
One decode attention step	$O(H \cdot T \cdot d_h)$		

9. Common Mistakes

- Saying “the Transformer is $O(S^2)$ ” without separating layers/terms.
- Calling KV-cache memory quadratic.
- Assuming causal masking changes big-O to linear.
- Saying FlashAttention reduces dense exact attention FLOPs to linear.
- Ignoring generated tokens in context growth.
- Equating tokenizer words with tokens.
- Assuming runtime maximum length guarantees model quality or service feasibility.

10. Edge Cases / Special Cases

- Tokenization can turn short-looking text/code into many tokens.
- Special/system/tool tokens count toward the context window.
- Some runtimes truncate, reject, or evict context when limits are exceeded; semantics differ.
- Sliding-window layers may coexist with occasional global-attention layers.
- Prefix caching reduces repeated prefill for reused prefixes but not cache capacity while active.
- Packed sequences need boundary masks despite sharing one physical token array.
- RoPE scaling methods change position treatment and must match cache construction.

11. How to Explain in Interview

Sequence length affects each component differently: token-wise projections and MLPs scale linearly, dense prefill attention has quadratic query-key pairs, KV storage is linear, and each decode token performs a linear scan over current context. FlashAttention reduces intermediate IO, not dense pairwise FLOPs, while sliding-window or sparse attention changes the actual attention pattern.

12. Quick Revision Notes

- Distinguish prompt S , current context T , output length N , and maximum window.
- Dense prefill attention: $\Theta(S^2)$ pairs.
- KV cache: $\Theta(T)$ storage.

- Decode attention: $\Theta(T)$ per new token.
- Causal triangle remains quadratic.
- FlashAttention changes IO/materialization; sparsity/windowing changes pair count.
- Trap: model/runtimes have constants and other terms; big-O alone does not predict dominance.

13. Practice Tasks

1. Compute valid causal pair counts for $S=4, 8, 1024$.
2. Derive cumulative decode attention pairs for prompt P and output N .
3. Compare dense and window-4096 pair counts at $S=32k$.
4. Estimate cache growth per additional 1k tokens.
5. Build a spreadsheet of component FLOPs over S for one model configuration.

14. Final Cheat Sheet

Item	Remember
Core definition	Number of prompt/context token positions processed or retained.
Why it matters	Controls pairwise attention, cache size, decode scans, and batching feasibility.
Most asked	S^2 attention, linear cache, per-step decode, FlashAttention, windows.
Main comparison	Dense changes pair count quadratically; windowed/sparse patterns bound it.
One-line answer	"Longer context linearly grows KV and decode scans but quadratically grows dense prefill attention pairs."

Tensor Parallelism

1. Overview

Definition. Tensor parallelism (TP) partitions the tensors and computation *inside each layer* across multiple GPUs. Instead of putting a complete layer on one device, ranks hold shards of its weight matrices and collaborate on every layer.

TP is used when a model/layer does not fit on one GPU or when more aggregate compute/memory bandwidth is needed per inference. It reduces per-GPU weight memory but introduces frequent inter-GPU collectives. It works best within fast, high-bandwidth/low-latency interconnect domains. Interviewers ask it to test matrix partitioning, collective communication, head sharding, comparison with data/pipeline parallelism, and scaling limits.

2. Core Idea

Consider $Y=XW$ with $W: [D, N]$ on two GPUs.

Column-parallel linear: split output columns.

```

W = [W0 | W1]
GPU0: Y0 = XW0
GPU1: Y1 = XW1
Y = [Y0 | Y1]

```

Each GPU needs the input X , but produces an independent output-feature shard. QKV and MLP gate/up projections naturally use this pattern because heads/intermediate features can remain sharded for the next local operation.

Row-parallel linear: split the reduction/input dimension.

```

W = [W0]      X = [X0 | X1]
    [W1]
GPU0: Z0 = X0W0
GPU1: Z1 = X1W1
Y = Z0 + Z1    # all-reduce or reduce-scatter form

```

Output projection and MLP down projection naturally consume the preceding feature shards, then a collective sums partial outputs.

One common Transformer pattern is therefore:

```

replicated X
→ column-parallel QKV
→ local heads/attention
→ row-parallel output projection
→ all-reduce partial outputs

replicated X
→ column-parallel gate/up
→ local activation/gating
→ row-parallel down projection
→ all-reduce partial outputs

```

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
TP degree/world size	Number of ranks splitting a layer.	TP=8.	Per-rank weight/head dimensions and divisibility.
Column parallel	Partition output features/weight columns.	shard Q heads.	No reduction immediately if consumer stays sharded.

Subtopic	What it means and why it matters	Example	Common interview angle
Row parallel	Partition input/reduction features/weight rows.	shard W_O input.	Partial outputs require sum.
All-reduce	Sum contributions and deliver result to all ranks.	residual stream replicated.	Communication volume/latency.
Reduce-scatter/all-gather	Keep outputs sharded or reconstruct inputs in alternative layouts.	sequence/tensor sharding combinations.	Avoid unnecessary replication where graph allows.
Head sharding	Assign attention heads/KV heads to ranks.	32 heads across 4 GPUs.	GQA divisibility/replication complications.
Communication overlap	Schedule collectives while independent compute proceeds.	asynchronous NCCL stream.	Dependency limits; verify real overlap.
Topology awareness	Place TP ranks on fast links.	intra-node NVLink/NVSwitch.	Slow PCIe/network can erase gains.
Small-shard efficiency	Larger TP makes each rank's GEMMs narrower/smaller.	TP=16 on modest D .	Kernel efficiency and launch overhead degrade.
Cache sharding	Each rank stores K/V for the heads it owns.	per-rank H_{kv}/TP , if divisible.	Per-rank capacity and attention locality.

4. Real-World Example

A model with $D=8192, H=64, H_{kv}=8$ runs with $TP=8$. Each rank can own 8 query heads and 1 KV head. The QKV projection is column-sharded; rank-local attention uses only its local Q/K/V cache. The output projection is row-sharded: each rank multiplies its 8-head output shard by its weight shard and produces a partial D -wide result. An all-reduce sums those eight partial results before the residual stream continues.

If TP is increased beyond 8, KV heads no longer divide naturally. The runtime may replicate KV heads, use a different partition, or reject the configuration. Meanwhile, smaller per-rank GEMMs and more communication can reduce scaling efficiency.

5. Diagrams / Mental Models

```

      replicated X
        /       \
GPU0: XW_QKV0   GPU1: XW_QKV1   column parallel
    local heads   local heads
    local attention local attention
GPU0: partial Y0 GPU1: partial Y1   row parallel
        \       /
        all-reduce

```

|
replicated Y

Cost as TP degree rises	Direction
Per-GPU weight memory	decreases
Per-GPU head/feature work	decreases
Number/frequency of participants	increases
Collective latency sensitivity	increases
Chance of small inefficient GEMMs	increases

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is tensor parallelism?	Intra-layer partitioning of weight/activation tensors across GPUs.	Different from whole-layer or replica placement.	Calling it data parallelism.
2. Why use it?	Fit large layers/models and combine device compute/bandwidth.	Memory plus performance motive.	Assuming speedup is automatic.
3. What is column-parallel linear?	Split weight output columns; each rank computes an output-feature shard.	$W = [W_0, W_1]$ by column blocks.	Splitting batch rows.
4. What is row-parallel linear?	Split the reduction/input dimension; ranks compute partial output sums that must be combined.	$Y = \sum X_i W_i$.	Concatenating partial sums.
5. Why pair column and row parallel?	Keep intermediate features local between adjacent operations and pay a collective after the row-parallel projection.	Communication-efficient Transformer pattern.	All-gathering after column projection unnecessarily.
6. Where are collectives in a block?	Commonly after attention output projection and MLP down projection, subject to architecture/runtime.	Two major reduction points.	One collective only at end of model.
7. How are attention heads sharded?	Assign groups of Q/KV heads to ranks when dimensions permit.	Local attention and cache.	Splitting one head's d_h by default.

Question	Clear answer	Interviewer expects	Common mistake
8. What limits TP scaling?	Collective latency/bandwidth, smaller GEMMs, synchronization, topology, and partition divisibility.	Strong-scaling trade-offs.	Only model size.
9. TP vs data parallelism?	TP splits one model/layer; data parallelism replicates the model and splits requests/training samples.	Memory/communication difference.	Saying both shard weights identically.
10. Why prefer fast intra-node links?	TP communicates large activations frequently on the critical path of every layer.	Latency/bandwidth sensitivity.	Discussing only one-time model loading.

7. Deep-Dive Questions

1. **Derive the row-parallel reduction.** With $X=[X_0|X_1]$ and vertically stacked $W=[W_0;W_1]$, distributivity gives $XW=X_0W_0+X_1W_1$; ranks compute terms, then sum.
2. **Why might all-reduce cost hurt decode disproportionately?** Decode GEMMs are small, so fixed collective latency and synchronization form a larger fraction of each layer's time.
3. **How does GQA complicate TP?** H_{kv} can be smaller than TP degree or indivisible, forcing KV replication/grouped mapping while Q heads remain sharded.
4. **Can TP reduce KV-cache memory per GPU?** Yes when KV heads/cache are sharded; total cluster cache is similar before replication/metadata, but each rank stores its local portion.
5. **Why can communication overlap be limited?** The next dependent operation needs the reduced result; overlap requires genuinely independent work and sufficient separate compute/communication resources.

8. Comparison Tables

Strategy	What is split	Main benefit	Main cost
Tensor parallelism	Tensors within each layer	Fit/accelerate one layer/model	Frequent collectives
Pipeline parallelism	Groups of layers/stages	Fit model across devices with less frequent activation transfer	Pipeline bubbles/stage balance
Data parallelism	Requests/batches across model replicas	High throughput and simple request isolation	Full model per replica; training gradient sync
Expert parallelism	MoE experts	Scale expert capacity	Token routing/all-to-all/load balance
Column-parallel linear		Row-parallel linear	
Split output columns		Split input/reduction rows	
Produces feature shards		Produces partial sums	
Often used for QKV and MLP gate/up		Often used for W_0 and MLP down	

Column-parallel linear**Row-parallel linear**

Can feed a sharded consumer locally

Usually followed by reduction collective

9. Common Mistakes

- Calling every multi-GPU scheme tensor parallelism.
- Saying row-parallel outputs concatenate instead of sum.
- All-gathering column-parallel outputs even when the next operation can remain sharded.
- Ignoring collective cost/critical-path synchronization.
- Assuming heads/KV heads divide every TP degree.
- Expecting linear speedup as per-rank kernels shrink.
- Placing latency-sensitive TP ranks across slow links without analysis.

10. Edge Cases / Special Cases

- Nondivisible hidden/intermediate/head dimensions may require padding or uneven shards.
- $H_{kv} < TP$ can require KV replication across subsets of ranks.
- Bias handling differs: adding a full bias before an all-reduce can multiply it; implementations add/shard it at the correct stage.
- Dropout/random operations in training require coordinated semantics across shards.
- Failures are collective: one stalled rank can block all ranks.
- Quantized packed weights need shard boundaries compatible with quantization groups/layouts.
- Multi-node TP is possible but often more sensitive to network topology than pipeline/data parallel approaches.

11. How to Explain in Interview

Tensor parallelism shards each layer across GPUs. Column-parallel QKV and MLP expansion give each rank independent head/feature outputs; local work continues without gathering. Row-parallel output/down projections consume those shards and generate partial results that an all-reduce combines. It lowers per-GPU memory and work but frequent collectives and shrinking GEMMs limit scaling, especially in decode.

12. Quick Revision Notes

- TP = intra-layer tensor sharding.
- Column split → output feature shards.
- Row split → partial sums → collective reduction.
- Common pairs: QKV → local attention → W_O ; gate/up → activation → down.
- Head/KV-head divisibility matters.
- Fast interconnect is critical; decode is latency-sensitive.
- Trap: more GPUs can slow inference when communication exceeds saved compute.

13. Practice Tasks

1. Split a 4×6 matrix column-wise across two GPUs and write output shapes.
2. Split a compatible multiplication row-wise and verify partial outputs must be added.

3. Draw one Transformer block at $TP=4$ with collective locations.
4. Calculate per-rank Q/K/V heads for several H, H_{kv}, TP combinations.
5. Benchmark or model compute time plus all-reduce time as TP increases.

14. Final Cheat Sheet

Item	Remember
Core definition	Partition layer tensors and computation across cooperating GPUs.
Why it matters	Fits large models and aggregates device resources.
Most asked	Column vs row parallel, collectives, heads, scaling limits, DP/PP comparison.
Main comparison	Column split yields shards; row split yields partial sums.
One-line answer	"Shard expansion by output columns, keep local features, shard contraction by input rows, then reduce partial outputs."

End-to-End Interview Synthesis

One Transformer Block, with Shapes and Likely GPU Behavior

Assume decoder-only inference, $M=B \cdot S$, standard MHA for simple shapes, and pre-norm:

Stage	Main equation/shape	GPU view	Key bottleneck/question
Input	$X: [B, S, D]$	contiguous/packed activation rows	layout, dtype, residual lifetime
Normalize	$\text{norm}(X)$	reduction + element-wise	fusion and accumulation precision
QKV projection	$[M, D] @ [D, 3D]$	packed GEMM	weight reuse; MHA/GQA/MQA width
Position transform	RoPE on Q/K	element-wise pairs	fuse with layout/cache write
Scores	$[S, d_h] @ [d_h, T]$	batched/head GEMM tiles	pair count and score IO
Softmax	row-wise across T	max/sum reductions	stability and online fusion
PV	$[S, T] @ [T, d_h]$	weighted accumulation	V-cache reads; no P materialization
Concatenate	heads $\rightarrow [B, S, D]$	view/reorder/fused output layout	whether a physical copy occurs

Stage	Main equation/shape	GPU view	Key bottleneck/question
Output projection	$[M,D]@[D,D]$	GEMM + epilogue	residual fusion; TP all-reduce
Normalize	$\text{norm}(\text{residual})$	reduction + element-wise	fusion
MLP gate/up	$[M,D]@[D,2D_{\text{ff}}]$	packed GEMM	large weight traffic
SwiGLU	$\text{SiLU}(g)\odot u$	element-wise/fused	avoid intermediate HBM trip
MLP down	$[M,D_{\text{ff}}]@[D_{\text{ff}},D]$	GEMM + epilogue	residual fusion; TP all-reduce

A Strong 60-Second Answer

A Transformer block receives hidden states $X[B,S,D]$. It normalizes and projects each token into Q, K, and V, usually with one packed GEMM. Per head, it computes scaled QK^T , applies the causal mask and stable row-wise Softmax, then multiplies by V to obtain a weighted content vector. Efficient attention tiles and fuses these stages so the $S \times T$ scores and probabilities do not go to HBM. Head outputs are concatenated, projected by W_O , and added to the residual. A gated MLP then expands each token, applies SwiGLU, projects back, and adds another residual. Prefill processes many prompt tokens with large GEMMs and fills KV cache; decode processes one new token per active request, streams weights and cached K/V, and is often bandwidth-sensitive. Batching improves weight reuse, sequence length grows attention/cache cost, and tensor parallelism shards each layer but introduces collectives.

Final Cross-Topic Interview Traps

- QKV/MLP/output linear layers mix hidden features, not token positions.
- Raw QK^T entries are logits; Softmax creates normalized weights.
- KV cache removes historical recomputation, not the current scan of historical K/V.
- FlashAttention reduces IO/materialization for exact dense attention; it does not make dense pair count linear.
- Causal attention remains quadratic during full prefill.
- Decode is sequential across generated tokens but massively parallel within each GPU step.
- Larger batch improves throughput only until memory, queueing, or other bottlenecks dominate.
- Fusion and CUDA Graphs solve related but different overheads.
- Tensor-parallel row shards produce partial sums, not concatenated final outputs.
- Always state tensor shapes, dtype, phase, and memory boundary when making a performance claim.

References

These are primary papers and official documentation useful for deeper study:

1. Vaswani et al., [Attention Is All You Need](#) — scaled dot-product and multi-head attention.
2. Dao et al., [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#) — tiled exact attention and HBM IO analysis.
3. Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#) — paged KV-cache management and continuous serving.

4. Narayanan et al., [Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM](#) — composition of tensor, pipeline, and data parallelism.
5. NVIDIA, [CUDA Programming Guide: Programming Model](#) — GPU execution and memory hierarchy.
6. NVIDIA, [GPU Performance Background](#) — compute/memory timing and roofline fundamentals.
7. NVIDIA, [Matrix Multiplication Background](#) — GEMM dimensions and arithmetic intensity.
8. NVIDIA NeMo, [Parallelisms](#) — practical tensor/model-parallel terminology.
9. NVIDIA TensorRT-LLM, [Performance Tuning Guide](#) — serving configuration and workload-sensitive tuning.
10. PyTorch, [Scaled Dot Product Attention](#) — SDPA semantics and fused CUDA backend selection.